

IPOG — Instituto de Pós-Graduação e Graduação

Projeto Integrador • Área 03 — Banco de Dados

Modelagem e análise de dados de vendas

Northwind Traders • PostgreSQL e MongoDB

Entrega 3 — Relatório cumulativo

Modelagem, consultas equivalentes e avaliação de desempenho

Giovanne Espindola

Trabalho individual

Setembro de 2026

Sumário

Seção	Página
1. Introdução e objetivo	3
2. Compreensão do negócio	4
3. Modelo conceitual	5
4. Análise exploratória e qualidade dos dados	7
5. Abordagem híbrida	12
6. Implementação relacional e decisões de modelagem	13
7. Modelagem documental	16
8. Transformação e carga dos dados	18
9. Estratégia de indexação	21
10. Conferência da migração	22
11. Síntese da modelagem e transição para analytics	23
12. Contrato analítico e método	24
13. Views e procedimentos armazenados	26
14. Consultas analíticas e resultados	27
15. Recursos documentais e MapReduce	36
16. Comparação de sintaxe e complexidade	37
17. Análise de performance e otimização	39
18. Conclusões e reprodução	44
Referências	45
Apêndice A — Consultas SQL e pipelines completos	46
Apêndice B — Views, procedures e recursos documentais	67

1. Introdução e objetivo

A Northwind Traders é uma empresa fictícia de distribuição de alimentos usada em uma base didática de vendas. Seus registros permitem estudar clientes, produtos, pedidos, fornecedores e a organização da equipe comercial. Este projeto utiliza essa base para desenvolver e comparar uma solução relacional em PostgreSQL e uma solução orientada a documentos em MongoDB.

O objetivo de negócio é identificar como o valor dos pedidos se distribui entre produtos, categorias, clientes e períodos, além de avaliar informações de estoque e de envio. O objetivo técnico é representar esse domínio nos dois bancos, executar consultas equivalentes e analisar os resultados, a organização dos dados e o desempenho observado.

Este relatório cumulativo reúne as Entregas 1, 2 e 3. Abrange negócio, exploração, modelos relacional e documental, migração, 16 análises equivalentes, views, procedures, recursos documentais e avaliação de desempenho. Códigos e resultados estão incluídos no documento.

1.1 Orientação metodológica

O CRISP-DM organiza projetos de dados em seis fases relacionadas: compreensão do negócio, compreensão dos dados, preparação, modelagem, avaliação e implantação [1]. A aplicação dessa metodologia abrangeu a compreensão do domínio, a avaliação da qualidade da base, a preparação dos dados e a definição das estruturas de armazenamento.

Fase	Aplicação no projeto
Compreensão do negócio	Definir o contexto, as perguntas e os limites da análise.
Compreensão dos dados	Examinar tabelas, relacionamentos, valores, ausências e consistência.
Preparação dos dados	Ajustar tipos e regras e preparar a transformação para documentos.
Modelagem	Desenvolver os modelos relacional e documental.
Avaliação	Conferir a equivalência das consultas e medir o desempenho.
Implantação	Disponibilizar as estruturas de armazenamento e documentar os resultados.

O uso é adaptado a um projeto de banco de dados: a modelagem é de estruturas de armazenamento, sem treinamento de modelos estatísticos. As fases podem ser retomadas. Por exemplo, a ausência de custos de aquisição levou a limitar as perguntas financeiras ao valor dos pedidos, sem calcular lucro.

1.2 Base e ferramentas

Foi utilizada a adaptação do Northwind para PostgreSQL mantida no repositório [pthom/northwind_psql](#) [2], cuja cópia SQL está preservada no projeto. A análise usa PostgreSQL 16, consultas SQL e um notebook Jupyter em Python. O PostgreSQL permite trabalhar com relacionamentos, restrições e valores decimais, além de inspecionar os planos das consultas.

O MongoDB 7 foi escolhido como o banco documental previsto no projeto da disciplina [3]. A transformação dos registros foi realizada em SQL, e a carga documental utilizou o mongosh, ambiente de execução de comandos do MongoDB.

2. Compreensão do negócio

A empresa comercializa produtos de diferentes categorias, fornecidos por empresas cadastradas. Os clientes fazem pedidos registrados por funcionários. Cada pedido reúne um ou mais itens, com produto, quantidade, preço praticado e desconto. Uma transportadora é indicada no pedido. A equipe comercial também está vinculada a territórios agrupados em regiões.

A base representa o cadastro comercial, os pedidos e informações de envio. Ela não registra o recebimento de pagamento nem a confirmação de entrega ao cliente. Assim, o processo observado vai do registro do pedido às informações disponíveis sobre seu envio.

2.1 Informações que se pretende obter

Pergunta de negócio	Uso da informação
Quais categorias e produtos concentram o valor dos pedidos?	Identificar a participação de cada parte do catálogo.
Como variam o volume e o valor médio dos pedidos ao longo dos meses?	Acompanhar a evolução comercial dentro do período disponível.
Quais clientes compram mais e quais estão sem compras recentes?	Apoiar ações de relacionamento com clientes.
Quanto foi registrado por vendedor e por equipe?	Descrever a distribuição das vendas entre os responsáveis.
Quais produtos estão abaixo do ponto de reposição?	Identificar sinais de atenção no estoque cadastrado.
Qual é o intervalo entre o pedido e o envio por transportadora?	Examinar o tempo até o envio registrado, sem confundir-lo com prazo de entrega.

Essas perguntas orientaram a exploração da Entrega 1 e foram detalhadas nas 16 análises do capítulo 14.

2.2 Definições para interpretar os resultados

Valor do item: preço praticado no item $\times$ quantidade $\times$ (1 – desconto). O valor de um pedido é a soma de seus itens. Neste relatório, “valor dos pedidos” é o total após descontos, sem acrescentar frete. Ele não comprova recebimento financeiro. Os valores são apresentados em unidades monetárias da base, sem atribuir uma moeda que não está identificada nas tabelas analisadas.

Ticket médio: soma do valor dos pedidos dividida pelo número de pedidos. O cálculo agrupa os itens antes de calcular a média, para não tratar cada item como um pedido independente.

Recência: as análises usam como referência o último dia com pedido, 06/05/1998. Usar a data atual tornaria todos os clientes antigos e não ajudaria a distinguir seu comportamento no recorte.

Limites do negócio: não há custo de aquisição para calcular margem ou lucro, nem registros de pagamentos, devoluções ou cancelamentos. A ausência dessas tabelas não comprova que esses eventos nunca ocorreram. A segmentação demográfica não é viável com as tabelas vazias, mas a segmentação por recência, frequência e valor das compras continua possível.

3. Modelo conceitual

O diagrama apresenta 11 entidades do recorte de vendas. Os identificadores descrevem como cada entidade é reconhecida; tipos SQL, índices e detalhes de armazenamento ficam para o modelo lógico e físico. ITEM DO PEDIDO e ATUAÇÃO são entidades associativas.

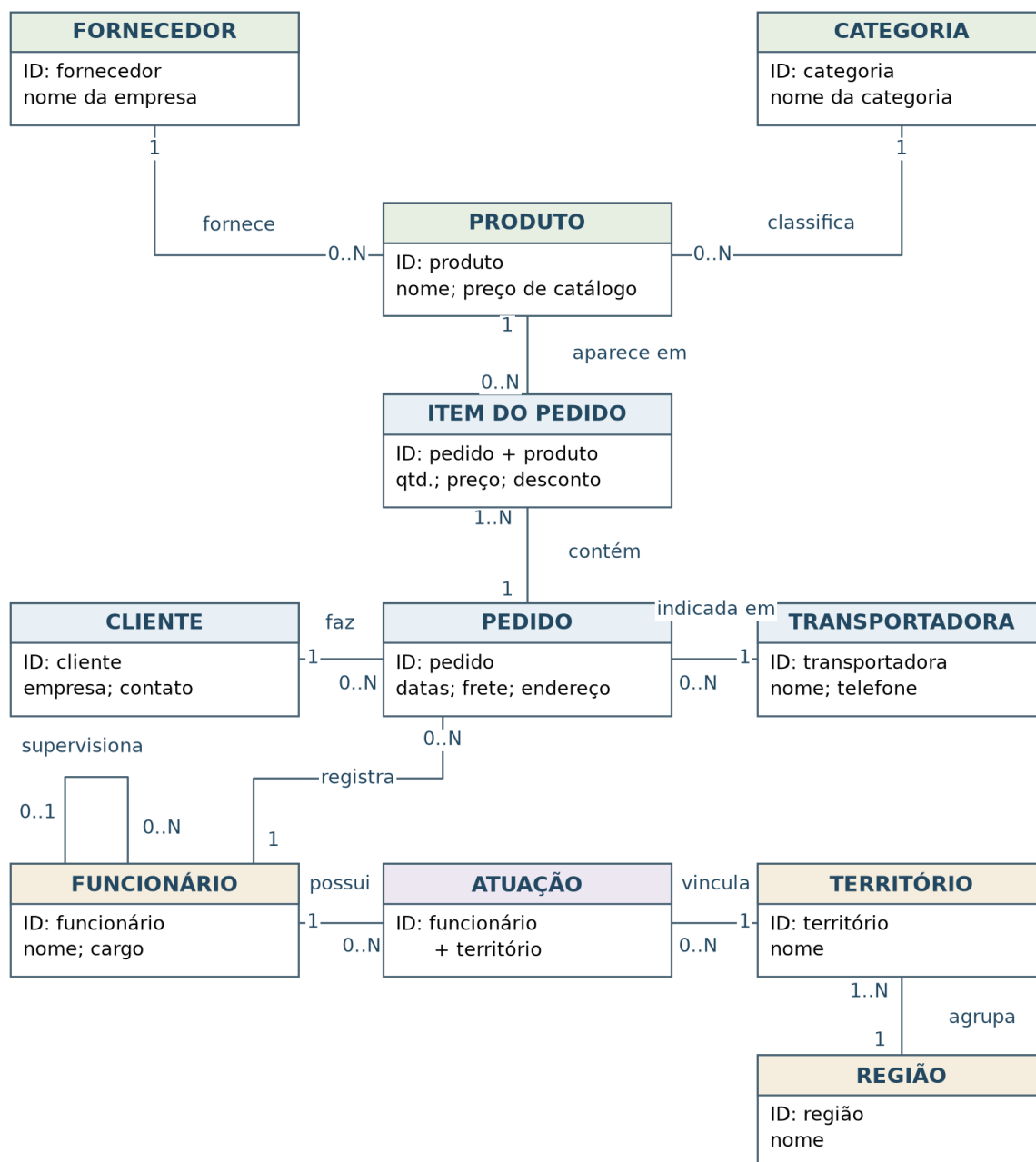


Figura 1 — Entidades e relacionamentos do recorte Northwind. Elaboração própria a partir da base [2]. Os números próximos a cada entidade indicam quantas ocorrências daquele lado podem se relacionar com uma ocorrência do outro lado: 1 = exatamente uma; 0..1 = opcional; 0..N = nenhuma ou várias; 1..N = uma ou várias.

3. Modelo conceitual — regras adotadas

As regras abaixo são escolhas para este projeto, compatíveis com os dados observados. A ausência de um caso na amostra não significa que ele deva ser proibido no modelo.

Relacionamento	Regra adotada
Cliente e pedido	Cada pedido tem um cliente. Um cliente pode ter zero ou vários pedidos.
Funcionário e pedido	Cada pedido tem um funcionário responsável. Um funcionário pode não ter pedidos.
Transportadora e pedido	Cada pedido tem uma transportadora indicada. Ela pode não aparecer em nenhum pedido.
Pedido e item	Um pedido tem um ou vários itens. Cada item pertence a um único pedido.
Produto e item	Cada item se refere a um produto. Um produto pode não aparecer em pedidos.
Categoria e produto	Cada produto pertence a uma categoria. A categoria pode não ter produtos.
Fornecedor e produto	Cada produto tem um fornecedor cadastrado. O fornecedor pode não ter produtos.
Região e território	Cada território pertence a uma região. Neste recorte, uma região reúne um ou vários territórios.
Funcionário, atuação e território	Cada atuação vincula um funcionário a um território. Ambos podem ter zero ou vários vínculos.
Supervisão de funcionários	Um funcionário tem no máximo um superior e pode supervisionar vários funcionários.

3.1 Identificação e participação

ITEM DO PEDIDO é identificado pelo par **pedido e produto**. O modelo não permite repetir esse par; a quantidade registra quantas unidades foram incluídas. ATUAÇÃO é identificada pelo par **funcionário e território**, permitindo representar vários responsáveis por um território quando necessário.

Na amostra, os 49 vínculos de atuação abrangem 49 territórios distintos: nenhum tem dois responsáveis registrados. A estrutura N:N é mantida como possibilidade do modelo. Também existem dois clientes sem pedidos, três transportadoras sem pedidos e quatro territórios sem funcionário vinculado.

As ligações de pedido com cliente, funcionário e transportadora, e de produto com categoria e fornecedor, aceitam nulos na fonte, embora não apresentem nulos na amostra. Sua obrigatoriedade é uma regra adotada no modelo do projeto. Já a exigência de ao menos um item por pedido precisa de validação adicional: uma chave estrangeira, sozinha, não garante que o pedido tenha itens.

3.2 Recorte do modelo

As tabelas **customer_demographics** e **customer_customer_demo** estão vazias e não são usadas nas análises. **us_states** tem 51 registros, mas não participa das relações utilizadas. As três permanecem preservadas na fonte. Sua exclusão se deve ao escopo; uma entidade pode ser válida mesmo sem registros.

4. Análise exploratória e qualidade dos dados

4.1 Procedimento e inventário

A conferência foi realizada em 05/09/2026, diretamente no PostgreSQL, em uma transação somente leitura. O notebook existente foi revisado e seus resultados estruturais foram conferidos por consultas independentes: contagem exata de linhas, inspeção das 92 colunas, nulos, chaves, relações e faixas de valores. Os comandos selecionados e suas saídas aparecem ao longo desta seção.

A análise usa o schema **public**, que preserva a fonte. O schema **nw** contém a implementação relacional, com 11 tabelas e 3.311 linhas. Esse schema foi utilizado como origem da transformação documental. Separar os schemas permite comparar a origem com o modelo proposto.

Tabela	Linhas	Colunas
categories	8	4
customer_customer_demo	0	2
customer_demographics	0	2
customers	91	11
employee_territories	49	2
employees	9	18
order_details	2155	5
orders	830	14
products	77	10
region	4	2
shippers	6	3
suppliers	29	12
territories	53	3
us_states	51	4

A soma é **3.362 linhas em 14 tabelas**. Esse número reúne tipos diferentes de registro; não corresponde ao número de vendas. Há 830 pedidos e 2.155 itens, com média de 2,60 itens distintos por pedido e intervalo de 1 a 25 itens.

Consulta 1 — Período e clientes com pedido.

```
SELECT min(order_date) AS primeiro_pedido,
       max(order_date) AS ultimo_pedido,
       count(*) AS pedidos,
       count(DISTINCT customer_id) AS clientes_com_pedido
FROM public.orders;
```

primeiro_pedido	ultimo_pedido	pedidos	clientes_com_pedido
1996-07-04	1998-05-06	830	89

O recorte vai de 04/07/1996 a 06/05/1998. Dos 91 clientes cadastrados, 89 aparecem em pedidos. Os demais podem ser estudados como clientes sem compra registrada nesse período.

4. Análise exploratória — valores e concentração

4.2 Valor e distribuição dos pedidos

Consulta 2 — Total e distribuição por pedido. Os campos são convertidos para decimal antes da multiplicação. O arredondamento é feito na apresentação do resultado, após a soma.

```
WITH totais AS (
  SELECT order_id,
         sum(unit_price::numeric * quantity *
              (1 - discount::numeric)) AS valor
  FROM public.order_details GROUP BY order_id
)
SELECT round(sum(valor), 2) AS total,
       round(avg(valor), 2) AS media,
       round(percentile_cont(0.5) WITHIN GROUP
              (ORDER BY valor)::numeric, 2) AS mediana,
       round(min(valor), 2) AS minimo,
       round(max(valor), 2) AS maximo
FROM totais;
```

total	media	mediana	minimo	maximo
1.265.793,04	1.525,05	943,25	12,50	16.387,50

O total é **1.265.793,04**, com ticket médio de **1.525,05** e mediana de **943,25**. A média maior que a mediana e o maior pedido de 16.387,50 indicam a influência de pedidos de valor elevado. Por isso, a mediana complementa a média na descrição do pedido típico.

4.3 Participação das categorias

Consulta 3 — Valor agrupado pela categoria do produto.

```
SELECT c.category_name AS categoria,
       round(sum(d.unit_price::numeric * d.quantity *
                 (1 - d.discount::numeric)), 2) AS valor
FROM public.order_details d
JOIN public.products p ON p.product_id = d.product_id
JOIN public.categories c ON c.category_id = p.category_id
GROUP BY c.category_name ORDER BY valor DESC;
```

categoria	valor
Beverages	267.868,18
Dairy Products	234.507,29
Confections	167.357,23
Meat/Poultry	163.022,36
Seafood	131.261,74
Condiments	106.047,09
Produce	99.984,58
Grains/Cereals	95.744,59

Beverages e Dairy Products somam **502.375,47**, aproximadamente **39,7%** do valor total. Essa concentração ajuda a identificar a participação comercial das categorias. Ela não mede lucro, pois os custos dos produtos não estão disponíveis.

4. Análise exploratória — período e dados ausentes

4.4 Distribuição temporal

ano	pedidos	valor
1996	152	208.083,97
1997	408	617.085,20
1998	270	440.623,87

Os pedidos foram agrupados pelo ano de `order_date`, com contagem dos pedidos e soma dos valores de seus itens. **1996 e 1998 são anos parciais**: os registros de 1996 começam em julho; os de 1998 terminam em maio. Comparar seus totais com 1997 como se fossem anos completos produziria uma interpretação inadequada de crescimento. Por isso, as análises da Entrega 3 usam séries mensais e trimestrais com os períodos parciais sinalizados; a janela curta limita conclusões sobre sazonalidade.

4.5 Completude

Foram contados os nulos de cada coluna por meio de `count(*) - count(coluna)`. Onze das 92 colunas apresentam pelo menos um nulo. As tabelas vazias não permitem avaliar completude de registros.

Coluna	Nulos	Total	%
customers.region	60	91	65,93
customers.postal_code	1	91	1,10
customers.fax	22	91	24,18
employees.region	4	9	44,44
employees.reports_to	1	9	11,11
orders.shipped_date	21	830	2,53
orders.ship_region	507	830	61,08
orders.ship_postal_code	19	830	2,29
suppliers.region	20	29	68,97
suppliers.fax	16	29	55,17
suppliers.homepage	24	29	82,76

Os nulos se concentram em informações de contato e endereço. Sem documentação adicional, não é possível distinguir automaticamente ausência de preenchimento de campo não aplicável. Esses nulos não foram substituídos por valores inventados. O nulo em **employees.reports_to** representa o funcionário sem superior cadastrado, no topo da hierarquia observada.

Consulta 4 — Data de envio e intervalo até o envio.

```
SELECT count(*) AS pedidos,
       count(*) FILTER (WHERE shipped_date IS NULL) AS sem_data,
       round(100.0 * count(*) FILTER
         (WHERE shipped_date IS NULL) / count(*), 2) AS pct_sem_data,
       round(avg(shipped_date - order_date), 2) AS dias_ate_envio
FROM public.orders;
```

pedidos	sem_data	pct_sem_data	dias_ate_envio
830	21	2,53	8,49

Há **21 pedidos sem data de envio registrada (2,53%)**. A média de 8,49 dias considera apenas os 809 pedidos com data de envio. Nenhum desses números informa quando o cliente recebeu a mercadoria ou comprova que os pedidos sem data nunca foram enviados.

4. Análise exploratória — integridade e validade

4.6 Identificadores e relacionamentos

A fonte possui **14 chaves primárias** e **13 chaves estrangeiras**. A revisão não encontrou duplicidade de identificadores nas 14 tabelas nem referências órfãs nas 13 relações verificadas. Isso não descarta duplicidades de cadastro com identificadores diferentes, que exigiriam critérios adicionais de comparação de nomes e endereços.

Consulta 5 — Exemplos de verificação de relacionamentos.

```
SELECT 'pedido sem cliente válido' AS verificacao, count(*) AS casos
FROM public.orders o WHERE NOT EXISTS (
  SELECT 1 FROM public.customers c WHERE c.customer_id = o.customer_id)
UNION ALL
SELECT 'item sem pedido válido', count(*)
FROM public.order_details d WHERE NOT EXISTS (
  SELECT 1 FROM public.orders o WHERE o.order_id = d.order_id)
UNION ALL
SELECT 'pedido sem item', count(*)
FROM public.orders o WHERE NOT EXISTS (
  SELECT 1 FROM public.order_details d WHERE d.order_id = o.order_id);
```

verificacao	casos
pedido sem cliente válido	0
item sem pedido válido	0
pedido sem item	0

Os três testes retornaram zero. A presença de itens em todos os pedidos também permite calcular o ticket usando os 830 pedidos da base, sem excluir pedidos vazios.

4.7 Faixas de valores e coerência temporal

Consulta 6 — Valores fora das faixas adotadas e envio anterior ao pedido.

```
SELECT 'quantidade <= 0' AS verificacao, count(*) AS casos
FROM public.order_details WHERE quantity <= 0
UNION ALL
SELECT 'preço < 0', count(*) FROM public.order_details WHERE unit_price < 0
UNION ALL
SELECT 'desconto fora de [0, 1)', count(*)
FROM public.order_details WHERE discount < 0 OR discount >= 1
UNION ALL
SELECT 'envio anterior ao pedido', count(*)
FROM public.orders WHERE shipped_date < order_date;
```

verificacao	casos
quantidade <= 0	0
preço < 0	0
desconto fora de [0, 1)	0
envio anterior ao pedido	0

Os preços dos itens variam de **2,00 a 263,50**, as quantidades de **1 a 130** e os descontos de **0% a 25%**. Não foram encontrados casos fora das regras testadas. O desconto inferior a 100% é uma regra deste projeto; o máximo observado de 25% não foi transformado em limite obrigatório.

A fonte não possui restrições **CHECK**, apesar de já ter chaves e outras regras de integridade. O modelo proposto usa validações explícitas para impedir valores inválidos em novas cargas. Ausência de erro na amostra e existência de uma proteção no banco são verificações diferentes.

4. Análise exploratória — precisão e histórico

4.8 Valores decimais e arredondamento

Na fonte, preço e desconto dos itens são armazenados como **real**, um tipo aproximado. A documentação do PostgreSQL recomenda **numeric** para valores monetários que exigem exatidão [4]. A conversão permite calcular sobre valores decimais, mas não recupera uma precisão que a origem já tenha perdido.

A soma executada com ponto flutuante foi **1265793.038653364**; com conversão para decimal, **1265793.0395**. Ambas são apresentadas como 1.265.793,04 com duas casas decimais. Há outra diferença, independente do tipo: o momento do arredondamento.

Consulta 7 — Arredondar a soma ou arredondar cada item.

```
SELECT round(sum(unit_price::numeric * quantity *
              (1 - discount::numeric)), 2) AS arredondar_total,
       sum(round(unit_price::numeric * quantity *
              (1 - discount::numeric), 2)) AS arredondar_itens
FROM public.order_details;
```

arredondar_total	arredondar_itens
1.265.793,04	1.265.793,29

A diferença é **0,25**. Este relatório usa **arredondamento apenas no total**. As views do schema nw criadas antes da Entrega 3 arredondam por item; por isso, as análises equivalentes passaram a usar as views de nw_analytics, sem arredondamento intermediário (capítulo 12). O valor maior não indica, por si só, perda de dados na migração.

4.9 Preço e endereço registrados no pedido

Consulta 8 — Comparação do preço do item com o cadastro do produto.

```
SELECT count(*) AS itens,
       count(*) FILTER
         (WHERE d.unit_price <> p.unit_price) AS precos_diferentes
FROM public.order_details d
JOIN public.products p ON p.product_id = d.product_id;
```

itens	precos_diferentes
2155	662

Em **662 dos 2.155 itens (30,7%)**, o preço praticado difere do preço do catálogo. A diferença não comprova quando ou por que o preço mudou. Ela mostra por que o cálculo deve usar o preço do item, preservado no pedido, em vez de substituí-lo pelo preço cadastrado do produto.

O mesmo cuidado se aplica ao endereço. Ao comparar destinatário, endereço, cidade, região, CEP e país, os seis campos coincidem com o cadastro do cliente em **748 dos 830 pedidos**; em 82 há diferença. A igualdade foi comparada tratando nulos correspondentes como iguais. O endereço do pedido foi preservado como informação daquele pedido, sem eliminar campos apenas porque repetem o cadastro atual.

4.10 Avaliação da base

A base é adequada às análises descritivas propostas: os relacionamentos conferem e não há violações nas faixas testadas. Os cuidados principais são tratar nulos conforme seu significado, manter os dados próprios do pedido, padronizar decimais e arredondamento e respeitar as limitações temporais e de cobertura do negócio.

5. Abordagem híbrida

O fluxo implementado é **PostgreSQL public → PostgreSQL nw → MongoDB**. A fonte original permanece preservada. O schema nw aplica as decisões relacionais; o MongoDB recebeu uma cópia para consultas, carregada por um processo em lote. A atualização documental foi definida em um único sentido, tendo o modelo relacional como origem.

5.1 Organização do MongoDB

Coleção	Organização implementada
orders	Um documento por pedido, com itens embutidos, datas, frete, endereço de envio e identificadores de cliente, funcionário e transportadora. Nos itens: produto, categoria, quantidade, preço praticado e desconto.
products	Produto com estoque, categoria e fornecedor identificados; nomes selecionados foram copiados para facilitar a leitura.
customers	Cadastro do cliente com endereço agrupado em um objeto.
employees	Cadastro do funcionário, referência ao superior e lista de territórios de atuação.
categories, suppliers e shippers	Cadastros completos para preservar também registros sem pedido ou produto associado.
territories e regions	Cadastros completos, incluindo territórios sem funcionário vinculado.

Foram implementadas **nove coleções**. Os 830 pedidos e seus 2.155 itens formam 830 documentos em orders. O critério de agrupamento segue a orientação de considerar o acesso aos dados ao decidir entre incorporação e referências [5].

5.2 Carga e validação

O processo usa identificadores estáveis e substituição por identificador, evitando duplicações ao repetir a carga. Registros removidos da origem provocam interrupção para revisão antes da gravação; não há exclusão automática. Valores monetários são convertidos de **numeric para Decimal128**, sem passar por float; datas e nulos seguem uma convenção única [6].

Os dados copiados de cadastros representam o estado disponível na extração, sem serem apresentados como histórico da data da venda. O preço praticado e o endereço de envio vêm do próprio pedido. A duplicação de nomes e categorias exige atualização das cópias quando a fonte mudar.

A validação compara identificadores, contagens, quantidade de itens por pedido e valores calculados com a mesma regra. Foram definidos validadores de estrutura e índices conforme as consultas. Referências entre coleções são conferidas após a carga; a validação de documentos não substitui as chaves estrangeiras relacionais [7].

5.3 Comparação entre os modelos

A incorporação dos itens foi avaliada por consultas equivalentes. A Entrega 3 acrescenta 16 pares analíticos, validação dos resultados e medições com aquecimento e vinte repetições. Os capítulos finais apresentam o desempenho observado, a dispersão e os limites da comparação.

6. Implementação relacional e decisões de modelagem

O schema **public** conserva a fonte com 14 tabelas. O schema **nw** representa o recorte implementado, com 11 tabelas e 3.311 linhas. Duas tabelas vazias de segmentação demográfica e a tabela `us_states` permanecem somente na fonte, pois não participam das análises. Essa separação permite verificar as alterações de tipos, estrutura e regras.

A implementação inclui 11 chaves primárias, 11 chaves estrangeiras, 17 restrições CHECK e quatro restrições UNIQUE. Os valores monetários usam `numeric(10,2)`, descontos usam `numeric(4,3)` e `discontinued` é booleano. A chave de `order_items` é (`order_id`, `product_id`): a quantidade representa as unidades do produto naquele pedido.

As regras verificam, entre outros aspectos, valores não negativos, descontos menores que um e coerência das datas. Chaves estrangeiras garantem a existência das referências; não garantem, sozinhas, que todo pedido tenha ao menos um item. Essa participação mínima é verificada separadamente.

6.1 Normalização e preservação da venda

Os itens ficam em uma relação própria, evitando colunas repetidas como `produto_1` e `produto_2`. O preço praticado depende do item daquela venda, não apenas do produto: usar o preço atual do cadastro alteraria o histórico. A exploração identificou 662 itens cujo preço difere do catálogo.

O cadastro do cliente fica separado dos pedidos. O endereço de envio permanece no pedido porque é uma informação daquela operação: 82 pedidos divergem do cadastro em pelo menos um dos seis campos comparados. Não se deve substituir esse endereço pelo atual.

O campo `quantity_per_unit` permanece como descrição da embalagem. Sua atomicidade depende do domínio adotado: tratado como descrição textual, ele é um atributo único; analisar separadamente quantidade, embalagem e unidade exigiria modelagem e transformação adicionais. Não se afirma BCNF para todas as tabelas apenas pela ausência de anomalias na amostra. Dependências funcionais precisam corresponder às regras do domínio.

6.2 Modelo lógico e índices

O diagrama lógico abaixo documenta tabelas, colunas, tipos e relações do schema `nw`. A consulta analítica acrescenta views em outro schema, sem mudar essas entidades.

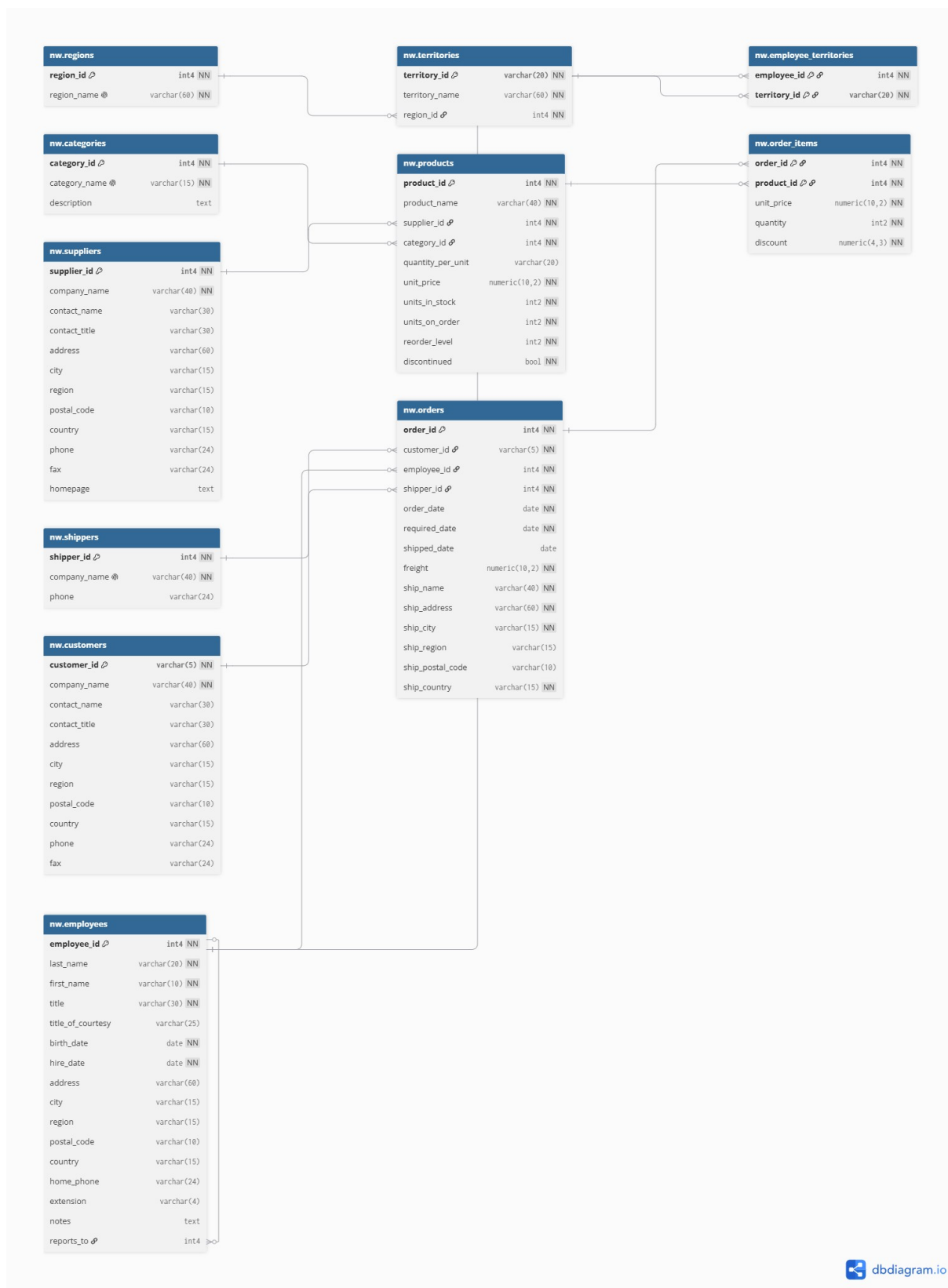


Figura 2 — Modelo lógico relacional de nw, mantido no projeto.

Existem quatro índices adicionais voltados aos acessos em `orders(customer_id)`, `orders(employee_id)`, `orders(order_date)` e `order_items(product_id)`. PKs e restrições UNIQUE também criam índices. As decisões anteriores permanecem contextualizadas ao volume da base; na Entrega 3, novos índices foram avaliados em cópias isoladas e não aplicados automaticamente ao banco original.

A escolha de um índice no plano depende da consulta, seletividade, estatísticas e volume. Uma varredura sequencial pode ser adequada numa tabela pequena. O relatório de performance apresenta os casos medidos e evita tratar a presença de índice como garantia de melhora.

6.3 Critérios para a modelagem documental

A exploração identificou uma base com 830 pedidos, 2.155 itens, 91 clientes e 77 produtos. Os pedidos cobrem parte de 1996, todo o ano de 1997 e parte de 1998. Seus valores apresentam concentração em determinadas categorias e diferença entre média e mediana, o que orienta o aprofundamento das análises comerciais.

As verificações de identificadores, referências e faixas de valores não apontaram inconsistências nas regras examinadas. As principais limitações são as datas de envio ausentes, a falta de custos e pagamentos, os anos parciais e o uso de tipos aproximados na fonte. Esses achados fundamentaram os critérios adotados na transformação dos dados.

6.3.1 Preservação das informações da venda

Os preços praticados e os endereços de envio foram mantidos como atributos próprios dos pedidos. A diferença entre o preço do item e o preço de catálogo em 662 itens mostrou que o cadastro atual não poderia substituir a informação da venda. Da mesma forma, as diferenças de endereço em 82 pedidos justificaram conservar o endereço de envio integralmente.

Os dados ausentes permaneceram nulos. A ausência de data de envio, por exemplo, não foi convertida em uma data estimada nem interpretada como comprovação de que o pedido nunca foi enviado. Os cadastros sem movimento também foram preservados, permitindo distinguir ausência de compra registrada de ausência do cliente na base.

6.3.2 Consistência dos indicadores

O cálculo dos valores manteve o preço praticado no item, a quantidade e o desconto, com arredondamento somente após a soma. A conversão para tipos decimais evitou introduzir novas aproximações de ponto flutuante na transformação, respeitando a precisão disponível na origem.

A conferência do modelo documental foi orientada pela comparação de identificadores e conteúdo, além das contagens. Esse critério permite verificar se a reorganização das linhas em objetos e arrays preservou as informações utilizadas nos indicadores. A igualdade de contagens, isoladamente, não comprovaria a manutenção de preços, descontos, datas ou endereços.

As regras de qualidade da base foram consideradas na definição dos validadores e na conferência das referências. A análise das consultas e do desempenho depende dessa consistência: diferenças decorrentes de arredondamento ou de perda de dados não devem ser interpretadas como características dos modelos de armazenamento.

7. Modelagem documental

7.1 Organização das coleções

A modelagem documental foi desenvolvida a partir das 11 tabelas do schema **nw**, preservando seus atributos, identificadores e relacionamentos. Os dados foram reorganizados em nove coleções no MongoDB. O principal agrupamento ocorreu entre pedidos e itens: o cabeçalho e os produtos vendidos passaram a compor um único documento por pedido.

Coleção	Documentos	Organização dos dados
orders	830	Pedido com datas, frete, referências a cliente, funcionário e transportadora, endereço shipping e array items.
products	77	Produto com preço de catálogo, estoque, categoria e fornecedor identificados e nomeados.
customers	91	Cadastro do cliente com informações de contato e endereço agrupado em objeto.
employees	9	Cadastro do funcionário, endereço, referência ao superior e array de territórios.
categories	8	Identificador, nome e descrição da categoria.
suppliers	29	Cadastro do fornecedor com contato e endereço agrupado em objeto.
shippers	6	Identificador, nome e telefone da transportadora.
territories	53	Identificador e nome do território, com referência à região.
regions	4	Identificador e nome da região comercial.

Foram obtidos **1.107 documentos**. A diferença em relação às 3.311 linhas relacionais decorre da incorporação dos 2.155 itens nos pedidos e dos 49 vínculos de território nos funcionários. Os cadastros sem movimento foram mantidos, incluindo clientes sem compra e transportadoras sem pedido associado.

7.2 Incorporação e referências

Os itens foram incorporados porque pertencem a um pedido e são consultados junto com ele. O endereço de envio também foi agrupado no pedido, preservando a informação registrada na venda. Essa decisão considera as diferenças de endereço identificadas na análise exploratória e evita substituí-lo pelo cadastro atual do cliente.

Produtos, clientes, funcionários e transportadoras permaneceram em coleções próprias, pois participam de vários pedidos e possuem atributos independentes de uma venda. Seus identificadores estabelecem as referências entre documentos. A organização combina incorporação e referências conforme as formas de acesso aos dados [5].

Nos funcionários, o array territories reúne os territórios de atuação, enquanto reports_to conserva a referência ao superior. As coleções territories e regions preservam a malha comercial completa, incluindo territórios sem funcionário vinculado. Os pedidos foram mantidos em coleção própria, evitando um array crescente de compras no cadastro de cada cliente.

7. Modelagem documental — estrutura do pedido

7.3 Representação do pedido 10248

O pedido 10248 pertence ao cliente VINET e contém três produtos. No modelo relacional, seus dados estão distribuídos entre `orders` e `order_items`. No MongoDB, o endereço de envio e os itens estão incorporados ao documento do pedido. O trecho abaixo apresenta os campos principais e o primeiro item; os demais itens foram omitidos somente desta representação.

```
{
  _id: 10248,
  customer_id: "VINET",
  order_date: ISODate("1996-07-04T00:00:00Z"),
  freight: Decimal128("32.38"),
  shipping: {
    name: "Vins et alcools Chevalier",
    street: "59 rue de l'Abbaye",
    city: "Reims", region: null,
    postal_code: "51100", country: "France"
  },
  items: [{
    product: {
      _id: 11, name: "Queso Cabrales",
      category: { _id: 4, name: "Dairy Products" }
    },
    unit_price: Decimal128("14.00"),
    quantity: 12, discount: Decimal128("0.000")
  }]
}
```

Os três produtos são 11, 42 e 72, com quantidades 12, 10 e 5 e preços praticados de 14,00, 9,80 e 34,80. O valor dos itens é **440,00**, sem incluir o frete de 32,38. Os preços foram obtidos dos itens da venda, preservando a diferença em relação ao preço de catálogo.

O nome do produto e sua categoria foram copiados para os itens a partir dos cadastros disponíveis na extração. Essas informações facilitam a leitura do pedido, mas não constituem histórico do catálogo na data da venda. Uma atualização cadastral exige nova carga para atualizar as cópias.

7.4 Tipos e valores ausentes

Os identificadores numéricos e as quantidades foram armazenados como **Int32**. Os códigos de clientes e territórios permaneceram como **strings**, preservando possíveis zeros à esquerda. Preços, descontos e frete foram armazenados como **Decimal128**, e valores SQL NULL foram mantidos como **null**.

As datas foram convertidas para **BSON Date à meia-noite UTC**, conservando o dia registrado na origem. Esse horário é uma convenção de representação, pois o tipo `date` do PostgreSQL não registra a hora do evento. Campos opcionais, como região, CEP e data de envio, permaneceram presentes nos documentos, admitindo **null** conforme a origem.

8. Transformação e carga dos dados

8.1 Agrupamento dos registros relacionais

A extração foi realizada no PostgreSQL em uma transação **REPEATABLE READ**, mantendo a mesma visão da origem durante a leitura das coleções. Os documentos foram construídos com funções JSON, sem alterações nos registros de nw. A operação central consistiu em agrupar as linhas de order_items pelo identificador do pedido.

Consulta 9 — Agrupamento dos itens do pedido 10248.

```
SELECT o.order_id,
       jsonb_agg(jsonb_build_object(
         'product_id', i.product_id,
         'unit_price', i.unit_price,
         'quantity', i.quantity,
         'discount', i.discount
       ) ORDER BY i.product_id) AS items
FROM nw.orders o
JOIN nw.order_items i USING (order_id)
WHERE o.order_id = 10248
GROUP BY o.order_id;
```

A consulta retorna uma linha para o pedido, com seus três itens no array items. O conteúdo obtido é apresentado abaixo, com quebras de linha para facilitar a leitura.

```
{
  "order_id": 10248,
  "items": [
    {"product_id": 11, "unit_price": 14.00,
     "quantity": 12, "discount": 0.000},
    {"product_id": 42, "unit_price": 9.80,
     "quantity": 10, "discount": 0.000},
    {"product_id": 72, "unit_price": 34.80,
     "quantity": 5, "discount": 0.000}
  ]
}
```

A função **jsonb_build_object** formou os campos de cada item, e **jsonb_agg** reuniu os registros em um array ordenado por produto [12]. Na transformação completa, cada pedido recebeu também suas datas, frete, referências e endereço de envio. Os itens receberam o identificador e o nome do produto, além da categoria aninhada.

O mesmo princípio foi aplicado aos vínculos de território: os registros da relação entre funcionário e território foram agrupados por employee_id e incorporados ao funcionário. Os demais cadastros deram origem a documentos individuais em suas respectivas coleções.

8. Transformação — tipos e gravação

8.2 Preservação dos tipos BSON

O transporte utilizou **Extended JSON**, que permite representar tipos BSON não diferenciados pelo JSON comum [9]. Os valores monetários foram convertidos de numeric para uma representação decimal textual e interpretados como Decimal128 no MongoDB, sem passagem por ponto flutuante.

O preço 14,00 foi representado da seguinte forma:

```
{"$numberDecimal": "14.00"}
```

A leitura com **EJSON.parse**, configurada com `relaxed: false`, converteu os marcadores de decimais, inteiros e datas nos respectivos tipos BSON. As funções JSON do PostgreSQL preservaram os caracteres de nomes e endereços, incluindo apóstrofes, e mantiveram os valores nulos.

8.3 Gravação por identificador

A carga foi executada pelo mongosh, utilizando operações de substituição agrupadas em `bulkWrite`. Cada documento conservou o identificador da origem no campo `_id`. O comando abaixo apresenta a operação aplicada aos pedidos, em que documento representa o registro completo produzido na transformação.

```
db.orders.replaceOne(
  { _id: documento._id },
  documento,
  { upsert: true }
);
```

A opção **upsert** insere o documento quando o identificador ainda não existe. Quando existe, **replaceOne** substitui seu conteúdo, incluindo os objetos e arrays aninhados. Essa estratégia permite repetir a carga sem criar novos identificadores para os mesmos registros.

Execução	Documentos inseridos	Documentos modificados
Primeira carga	1.107	0
Repetição da mesma carga	0	0

O resultado da segunda execução confirmou a estabilidade da carga para o mesmo conjunto de dados. Alterações na origem exigem nova extração. A atualização é feita em lote, e identificadores existentes no destino que não aparecem na extração provocam interrupção antes das escritas, para revisão. Não foi adotada propagação automática de exclusões.

8.4 Limites da carga

O MongoDB foi utilizado em modo **standalone**, com atomicidade por documento, mas sem transação única envolvendo todas as coleções [8]. Uma interrupção durante a gravação pode deixar o lote incompleto. A recuperação adotada consiste na repetição integral da carga após corrigir a causa, seguida de nova conferência de integridade.

A rotina foi desenvolvida para o conjunto pequeno e estático do Northwind. Os documentos são lidos em memória, e coleções vazias são recusadas para revisão. A implementação não foi avaliada como mecanismo de sincronização contínua ou de migração de grandes volumes.

8. Validação da estrutura documental

8.5 Regras implementadas

As nove coleções receberam validadores de estrutura, aplicados também aos objetos e arrays aninhados. Foram definidos os campos obrigatórios, os tipos BSON e as faixas de valores. A validação foi configurada para rejeitar inserções e alterações incompatíveis com as regras [7].

Campo ou grupo	Regra adotada
Preços e frete	Decimal128, com valor maior ou igual a zero.
Quantidade do item	Inteiro, com valor maior ou igual a um.
Desconto do item	Decimal128, maior ou igual a zero e menor que um.
Datas	BSON Date; data de envio admite null.
Endereço de envio	Objeto obrigatório, com destinatário, logradouro, cidade, região, CEP e país; região e CEP admitem null.
Itens do pedido	Array de objetos com produto, categoria, preço praticado, quantidade e desconto.

O trecho a seguir apresenta a regra aplicada ao desconto dos itens, incluindo tipo, limites e exclusão do limite superior.

```
discount: {
  bsonType: "decimal",
  minimum: Decimal128("0"),
  maximum: Decimal128("1"),
  exclusiveMaximum: true
}
```

8.6 Testes de rejeição

Foram realizadas cinco tentativas de substituição de um pedido válido em uma base temporária isolada. Cada tentativa alterou apenas o aspecto indicado na tabela.

Alteração testada	Resultado
Preço armazenado como texto	Rejeitada: código 121.
Quantidade igual a zero	Rejeitada: código 121.
Desconto igual a 100%	Rejeitada: código 121.
Data do pedido armazenada como texto	Rejeitada: código 121.
Ausência do objeto de endereço de envio	Rejeitada: código 121.

Todas as alterações foram rejeitadas, e o documento válido permaneceu inalterado. O resultado demonstra a atuação dos validadores diante das inconsistências testadas, além da constatação de que os documentos carregados atendiam às regras.

Os validadores não substituem as chaves estrangeiras nem reproduzem todas as restrições do modelo relacional. A existência dos cadastros referenciados, a coerência temporal e a ausência de produtos repetidos dentro de um pedido foram verificadas separadamente, após a carga.

9. Estratégia de indexação

9.1 Índices associados a consultas

Além do índice único automático em `_id` de cada coleção, foram criados três índices adicionais. Eles atendem a formas concretas de acesso previstas para o modelo.

Índice	Campos	Consulta atendida
cliente_data	customer_id: 1, order_date: 1	Pedidos de um cliente em determinado período, ordenados por data.
produto_no_pedido	items.product._id: 1	Pedidos que contêm determinado produto.
categoria_produto	category._id: 1	Produtos cadastrados em uma categoria.

No índice composto, o cliente é usado por igualdade e a data permite o intervalo e a ordenação. O índice em `items.product._id` é **multikey**: o MongoDB cria entradas para valores encontrados no array de itens [10]. Não foi usado índice único nesse campo, porque o mesmo produto pode aparecer em vários pedidos.

```
db.orders.createIndex(
  { customer_id: 1, order_date: 1 },
  { name: "cliente_data" }
);
```

9.2 Evidência do plano de execução

A consulta abaixo foi executada com `explain`, sem forçar índice com `hint`:

```
db.orders.find({
  customer_id: "VINET",
  order_date: {
    $gte: ISODate("1996-01-01T00:00:00Z"),
    $lt: ISODate("1997-01-01T00:00:00Z")
  }
}).sort({ order_date: 1 }).explain("executionStats");
```

Medida observada	Resultado
Plano vencedor	FETCH → IXSCAN
Índice utilizado	cliente_data
Documentos retornados	3
Documentos examinados	3
Chaves examinadas	3

IXSCAN indica leitura pelo índice; **FETCH** indica a busca dos documentos correspondentes. A consulta retorna três pedidos de VINET em 1996. Esse resultado demonstra uso do índice nessa consulta, sem justificar uma conclusão geral sobre superioridade do MongoDB.

Índices consomem espaço e precisam ser mantidos durante as escritas. Por isso, não foram criados índices para todos os campos. A observação realizada sustenta o uso do índice composto para o filtro e a ordenação examinados. Os outros dois índices foram definidos pelos acessos previstos. Na Entrega 3, o plano de P03 mostrou o uso de `produto_no_pedido` pelo `$lookup` que procura vendas de cada produto; a comparação de desempenho entre os bancos está no capítulo 17.

10. Conferência da migração

10.1 Equivalência dos registros

A conferência foi realizada em 10/09/2026, mediante a leitura dos documentos gravados no MongoDB e a reconstrução das linhas das 11 tabelas de nw. Os itens foram separados dos pedidos, os campos dos endereços foram desagrupados e os vínculos entre funcionários e territórios foram reconstituídos. Esse procedimento foi desenvolvido separadamente da transformação utilizada na carga.

As linhas reconstruídas foram comparadas com a origem por **EXCEPT ALL nos dois sentidos**. A comparação abrangeu os campos e as ocorrências de cada linha, permitindo identificar registros ausentes, extras, duplicados ou com valores alterados. Os decimais foram comparados sem arredondamento prévio, e datas com horário diferente da convenção de meia-noite UTC foram recusadas antes da comparação.

Tabela reconstruída	Origem	MongoDB	Diferenças
regions	4	4	0
territories	53	53	0
categories	8	8	0
shippers	6	6	0
suppliers	29	29	0
customers	91	91	0
employees	9	9	0
employee_territories	49	49	0
products	77	77	0
orders	830	830	0
order_items	2.155	2.155	0
Total	3.311	3.311	0

As **3.311 linhas** foram reconstruídas sem divergências. Também foram conferidos os tipos BSON, as referências entre coleções, os nomes e categorias copiados para os itens, os territórios incorporados e a coerência das datas. A comparação foi repetida após a segunda carga, mantendo zero diferenças.

10.2 Precisão dos valores

A soma dos itens foi calculada separadamente nos dois bancos pela expressão **preço praticado × quantidade × (1 – desconto)**, sem inclusão do frete. PostgreSQL e MongoDB retornaram o mesmo valor decimal exato: **1265793.03950**. Com arredondamento apenas após a soma, o valor apresentado é **1.265.793,04**.

A conferência não utilizou as views que arredondam cada item. A comparação exata também evita depender das diferenças entre round(numeric) do PostgreSQL e \$round do MongoDB em casos de empate [4, 11].

Como teste adicional, uma cópia dos dados de retorno recebeu o preço 14.004 em lugar de 14.00. A comparação identificou duas diferenças: a ausência da linha original e a presença da linha alterada. O teste confirmou que uma diferença inferior a um centavo não foi ocultada por arredondamento. Nenhum registro dos bancos foi alterado nesse teste.

11. Síntese da modelagem e transição para analytics

A modelagem documental representou os dados relacionais do Northwind em nove coleções, com 1.107 documentos. A incorporação dos itens e do endereço de envio permitiu reunir os dados de cada pedido em uma única estrutura. Os cadastros permaneceram em coleções próprias, mantendo as referências necessárias e os registros sem movimento.

A transformação preservou os identificadores, os campos nulos, as datas, os preços praticados, os descontos e os endereços dos pedidos. A conferência independente reconstruiu as 3.311 linhas de nw sem divergências. A repetição da carga não inseriu nem modificou documentos, confirmando que o mesmo conjunto de dados pode ser carregado novamente sem duplicação.

11.1 Avaliação do modelo

O pedido 10248 exemplifica a organização adotada: seus três itens, antes armazenados em linhas separadas, passaram a integrar um array no documento. Produto e categoria foram representados em objetos aninhados, permitindo identificar os itens sem consultar outros cadastros para obter seus nomes. O custo dessa escolha é a manutenção das informações copiadas quando os cadastros mudam.

Os validadores rejeitaram as cinco alterações inválidas testadas e preservaram o documento original. A existência das referências e a coerência entre documentos foram verificadas separadamente, pois a estrutura documental não oferece as mesmas garantias das chaves estrangeiras relacionais.

Os índices foram definidos para pedidos por cliente e período, pedidos por produto e produtos por categoria. Na consulta observada para o cliente VINET em 1996, o MongoDB utilizou o índice composto e examinou três documentos para retornar três pedidos. Esse resultado sustenta a escolha do índice para o acesso avaliado.

11.2 Limites da etapa de modelagem

Na conclusão da Entrega 2, os resultados demonstravam a preservação dos dados, sem medições de tempo entre consultas equivalentes. A Entrega 3 acrescenta essas medições nos capítulos seguintes. O volume permanece pequeno e não permite extrapolar para escalabilidade.

A carga em lote depende de nova extração para refletir mudanças na origem. As cópias de nomes e categorias representam o estado cadastral disponível na extração, sem reconstruir seu histórico na data da venda. O modo standalone também limita a atomicidade ao documento, exigindo nova conferência após a recuperação de uma carga interrompida.

As análises seguintes implementam consultas equivalentes e avaliam desempenho, mantendo as definições dos indicadores, o preço praticado no item e os critérios de precisão. A equivalência já verificada fornece uma base consistente para comparar os dois modelos sem atribuir a diferenças de tecnologia resultados causados por perda ou alteração de dados.

12. Contrato analítico e método

A Entrega 3 responde a 16 perguntas com consultas SQL e pipelines MongoDB equivalentes. A comparação usa os dados do Northwind conferidos em 22/09/2026 e novamente em 24/09/2026: 830 pedidos, 2.155 itens, 91 clientes e 77 produtos. A reconstrução das 3.311 linhas relacionais foi repetida sem divergências. Os scripts completos estão nos apêndices deste documento.

12.1 Regras compartilhadas

O período padrão é **[01/07/1996, 01/06/1998]**: início incluído e fim excluído. Ele contém todos os pedidos disponíveis, de 04/07/1996 a 06/05/1998. As séries mensais incluem os 23 meses do intervalo. Julho de 1996 e maio de 1998 têm cobertura parcial dos registros.

Valor significa **preço praticado × quantidade × (1 – desconto)**, sem frete. Todos os cálculos monetários usam numeric ou Decimal128. Somatórios são comparados sem arredondamento; médias e percentuais admitem diferença absoluta máxima de 0,0000000000001, devido às divisões com expansão decimal. Numeradores e denominadores também são conferidos. Na exibição, os valores usam duas casas e ROUND_HALF_UP. As implementações não dependem de igual comportamento de round e \$round [4, 11].

O total exato é **1265793.03950**. Arredondar somente ao final resulta em **1.265.793,04**; somar itens previamente arredondados resulta em **1.265.793,29**. As views históricas permanecem preservadas. As novas consultas monetárias usam o schema **nw_analytics**, sem arredondamento intermediário.

Ticket é valor dos pedidos dividido pelo número de pedidos. Um pedido com vários itens conta uma única vez. Identificadores definem os grupos; nomes servem para apresentação. A ordenação final é explícita nos dois bancos e também foi validada.

12.2 Recência, ranking e limites

Inatividade e RFM consideram todo o histórico até **06/05/1998**, inclusive, independentemente da janela mensal. O corte de seis meses é **06/11/1997**. Última compra anterior ao corte significa inatividade; compra no próprio corte não. Clientes sem compras formam uma categoria própria.

O ranking retorna até cinco produtos por categoria, com valor decrescente e ID crescente no desempate. A curva ABC considera a participação acumulada anterior ao cliente: classe A abaixo de 80%, B abaixo de 95%, C nos demais casos. O cliente que cruza o limite completa a classe em que entrou. Clientes sem compras ficam fora dessas classes.

RFM recebe faixas didáticas fixas, sem ajuste preditivo. R: até 30 dias = 3, até 90 = 2, acima = 1. F: pelo menos 10 pedidos = 3, pelo menos 5 = 2, abaixo = 1. M: pelo menos 10.000 unidades monetárias = 3, pelo menos 5.000 = 2, abaixo = 1. Sem compras: R, F e M recebem zero; recência permanece nula. Essas faixas são uma convenção de segmentação, não um padrão universal nem recomendação validada de marketing.

Estoque é o estado cadastrado, sem reconstrução histórica. Data de envio ausente significa ausência de registro, não prova de que o pedido nunca saiu. O intervalo até envio não é tempo de entrega. A base não permite calcular lucro, margem ou recebimento financeiro. A distribuição trimestral não demonstra sazonalidade recorrente.

12.3 Contratos de saída

Cada arquivo QNN.sql tem um PNN.js correspondente. As consultas são executáveis com o recorte padrão escrito no código; alterar um parâmetro exige atualizar as duas pontas e repetir a conferência. Q05 usa o estoque cadastrado; Q06 e Q11 usam a referência histórica indicada acima.

As saídas monetárias são decimais; IDs, contagens, quantidades e dias são inteiros, embora somas SQL de contagens possam ser representadas por numeric. Datas de resultado usam AAAA-MM-DD. Valores indefinidos são null nos dois bancos. As tabelas do relatório mostram a mesma saída validada, com formatação aplicada somente depois da comparação.

Pares	Unidade de cada linha e ordenação
01 e 14	Categoria com movimento; ID da categoria crescente
02, 08 e 09	Mês do calendário; mês crescente, incluindo meses sem pedidos

Pares	Unidade de cada linha e ordenação
03	Produto sem venda no período; ID do produto crescente
04	Funcionário cadastrado, inclusive sem pedidos; ID crescente
05	Categoria com produto abaixo do ponto; ID da categoria crescente; lista de produtos ordenada
06	Cliente inativo ou sem compras; código crescente
07	Produto classificado dentro da categoria; categoria e posição crescentes
10	Todos os clientes; valor decrescente, código crescente
11	Todos os clientes; código crescente
12	Par de produtos com pelo menos uma compra conjunta; contagem decrescente, IDs crescentes
13	Transportadora cadastrada, inclusive sem pedidos; ID crescente
15	Funcionário considerado como responsável por sua própria equipe; ID crescente
16	Trimestre com movimento, total e categorias; ano/trimestre/ID, com total antes das categorias

12.4 Validação

Os 16 pares foram comparados linha a linha e campo a campo, sem reordenar os resultados depois da execução. A comparação também exige tipos compatíveis: inteiros e decimais são aceitos, mas ponto flutuante, valores lógicos trocados por números e datas fora da meia-noite UTC são rejeitados. Foram conferidos também os totais por mês, categoria e funcionário, as contagens de pedidos e a identidade bruto – desconto = valor líquido. O retorno vazio de produtos nunca vendidos foi preservado.

Como SQL e MongoDB poderiam compartilhar o mesmo erro de interpretação, os resultados também foram recalculados por um terceiro caminho, em Python, diretamente sobre as linhas de nw, com igualdade nos 16 pares.

Foram concluídas ainda **38 verificações adicionais** em dados sintéticos isolados: 16 pares com quatro pedidos de fronteira, 16 pares sem nenhum pedido, empate no ranking, empate de meio centavo no arredondamento, três rejeições de parâmetros inválidos e uma conferência conjunta das três procedures. Nos pedidos de fronteira, além da igualdade entre os bancos, 15 pares foram comparados com valores esperados calculados à mão, por exemplo: mês sem vendas com ticket nulo, percentual nulo após mês com valor zero, compra exatamente no corte de inatividade (fora da lista) e um dia antes (dentro), média de envio que ignora a data ausente e suporte de 25% para o único par em quatro pedidos. As estruturas temporárias foram removidas; os registros originais permaneceram inalterados.

13. Views e procedimentos armazenados

As quatro novas views compõem uma camada de relatórios em `nw_analytics`. `vw_item_valor` calcula bruto, desconto e valor exato por item; `vw_pedido_valor` reúne contagens e valor por pedido; `vw_vendas_mensais` resume os meses com movimento; `vw_envio_transportadora` apresenta pedidos, datas ausentes e intervalo até envio. O calendário completo é construído nas consultas temporais.

Uma view armazena a definição da consulta, não uma cópia materializada dos resultados. Sua finalidade é centralizar a regra de valor para que todas as análises a apliquem da mesma forma. `vw_item_valor` alimenta Q01, Q07, Q14 e Q16; `vw_pedido_valor` alimenta Q02, Q04, Q08 a Q11 e Q15. As outras duas podem ser consultadas diretamente como relatórios prontos, por exemplo `SELECT * FROM nw_analytics.vw_envio_transportadora`. As views antigas de `nw`, que arredondam por item, foram mantidas sem alteração e não são usadas nos valores da Entrega 3.

13.1 Procedures parametrizadas

Foram implementadas três procedures PL/pgSQL [13]. `sp_resumo_vendas_periodo` recebe início/fim e retorna, por categoria, pedidos distintos, valor e ticket. Um pedido com mais de uma categoria conta em cada categoria; não se somam essas contagens como se fossem pedidos distintos do conjunto. `sp_clientes_inativos` recebe a data de referência e o número de meses. `sp_desempenho_equipe` recebe gestor e período e retorna vendas próprias e da equipe.

Os relatórios tabulares usam um parâmetro INOUT refcursor. Esse parâmetro recebe o nome de um cursor: a procedure abre o cursor com o resultado, e o cliente o lê com `FETCH` na mesma transação, como no exemplo do Apêndice B. As três rotinas foram criadas com `CREATE PROCEDURE` e são executadas com `CALL`; no PostgreSQL, funções (`CREATE FUNCTION`) são objetos diferentes, chamados dentro de consultas.

Períodos invertidos, quantidade de meses não positiva e gestor inexistente são rejeitados. Os três resultados foram comparados com consultas independentes. Para o gestor 2, a procedure retorna nove membros, 830 pedidos e o valor total da base, porque toda a hierarquia está abaixo dele; os pedidos de cada membro entram uma única vez.

As definições completas das views, das procedures e das chamadas estão no Apêndice B. Nas saídas abaixo, os decimais foram arredondados para duas casas apenas na apresentação.

Saída de `CALL sp_resumo_vendas_periodo('1996-07-01', '1998-06-01', ...)`:

category_id	categoria	pedidos	valor	ticket_categoria
1	Beverages	354	267.868,18	756,69
2	Condiments	193	106.047,09	549,47
3	Confections	295	167.357,23	567,31
4	Dairy Products	303	234.507,29	773,95
5	Grains/Cereals	182	95.744,59	526,07
6	Meat/Poultry	161	163.022,36	1.012,56
7	Produce	129	99.984,58	775,07
8	Seafood	291	131.261,74	451,07

A chamada `CALL sp_clientes_inativos('1998-05-06', 6, ...)` devolveu 7 clientes, os mesmos de Q06, na mesma ordem (tabela completa na seção 14.6).

Saída de `CALL sp_desempenho_equipe(2, '1996-07-01', '1998-06-01', ...)`:

gestor_id	membros	valor proprio	pedidos equipe	valor equipe
2	9	166.537,76	830	1.265.793,04

14. Consultas analíticas e resultados

Cada seção apresenta a pergunta, a regra de cálculo, os recursos usados em cada banco, o resultado executado e sua interpretação. As tabelas reproduzem a saída validada, idêntica nos dois bancos; os cabeçalhos traduzem os nomes das colunas e os decimais são arredondados para duas casas somente na apresentação. Quando o resultado é extenso, o recorte exibido está indicado; a validação considerou todas as linhas. Os 32 códigos completos estão no Apêndice A.

14.1 Q01/P01 — Valor e participação por categoria

Pergunta: Quais categorias concentram o valor dos pedidos?

Cálculo: Relacionar itens ao pedido e à categoria, somar quantidade e valor e dividir cada valor pelo total geral.

Implementação: SQL: JOIN, GROUP BY e SUM OVER. MongoDB: \$unwind, \$group e \$setWindowFields.

Resultado: 8 linhas conferidas; todas exibidas.

categoria (ID)	categoria	unidades	valor	participação (%)
1	Beverages	9532	267.868,18	21,16
2	Condiments	5298	106.047,09	8,38
3	Confections	7906	167.357,23	13,22
4	Dairy Products	9149	234.507,29	18,53
5	Grains/Cereals	4562	95.744,59	7,56
6	Meat/Poultry	4199	163.022,36	12,88
7	Produce	2990	99.984,58	7,90
8	Seafood	7681	131.261,74	10,37

Interpretação: Beverages representa 21,16% do valor e soma 267.868,18. Participação comercial não é margem: o custo de aquisição não está disponível.

Limite de leitura: A participação mede peso no valor dos pedidos, não rentabilidade. Os grupos são definidos pelo ID da categoria e o percentual é calculado depois da agregação, sobre o mesmo recorte.

14.2 Q02/P02 — Pedidos e ticket médio mensal

Pergunta: Como variam o número de pedidos e o ticket médio mensal?

Cálculo: Somar itens por pedido, agrupar pedidos por mês e completar o calendário. Ticket = valor / pedidos.

Implementação: SQL: CTEs, generate_series e agregação em dois níveis. MongoDB: \$map, \$group, \$facet e calendário por \$dateAdd.

Resultado: 23 linhas conferidas; todas exibidas.

mês	pedidos	valor	ticket
1996-07-01	22	27.861,90	1.266,45
1996-08-01	25	25.485,28	1.019,41
1996-09-01	23	26.381,40	1.147,02
1996-10-01	26	37.515,73	1.442,91
1996-11-01	25	45.600,05	1.824,00
1996-12-01	31	45.239,63	1.459,34
1997-01-01	33	61.258,07	1.856,31
1997-02-01	29	38.483,64	1.327,02
1997-03-01	30	38.547,22	1.284,91
1997-04-01	31	53.032,95	1.710,74
1997-05-01	32	53.781,29	1.680,67
1997-06-01	30	36.362,80	1.212,09

mês	pedidos	valor	ticket
1997-07-01	33	51.020,86	1.546,09
1997-08-01	33	47.287,67	1.432,96
1997-09-01	37	55.629,24	1.503,49
1997-10-01	38	66.749,23	1.756,56
1997-11-01	34	43.533,81	1.280,41
1997-12-01	48	71.398,43	1.487,47
1998-01-01	55	94.222,11	1.713,13
1998-02-01	54	99.415,29	1.841,02
1998-03-01	73	104.854,16	1.436,36
1998-04-01	74	123.798,68	1.672,96
1998-05-01	14	18.333,63	1.309,55

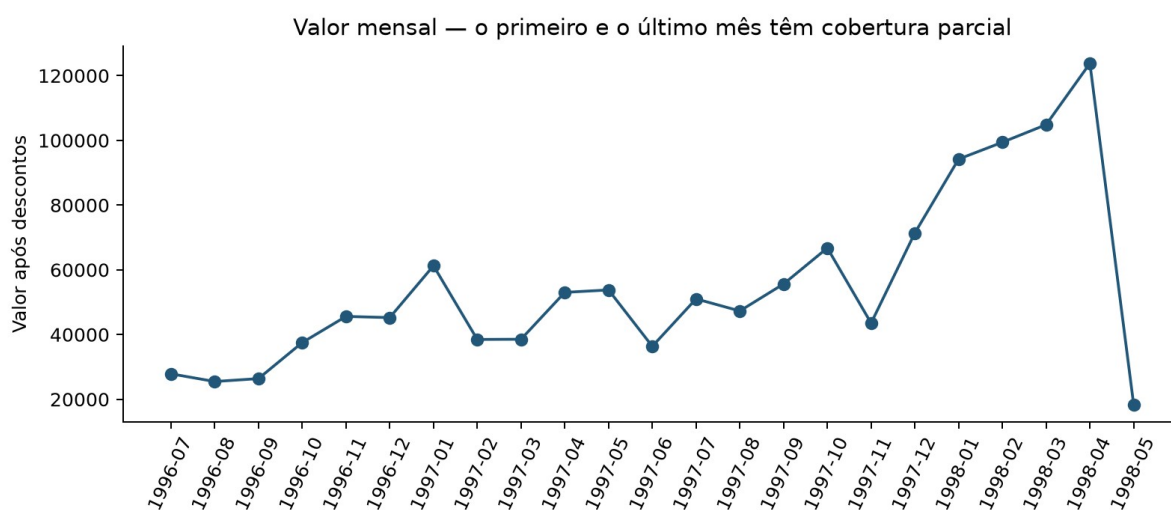


Figura 3 — Valor mensal após descontos (Q02/P02). Julho de 1996 começa no dia 4 e maio de 1998 termina no dia 6.

Interpretação: Julho de 1996 tem 22 pedidos, valor de 27.861,90 e ticket de 1.266,45. O maior valor mensal observado é abril de 1998, com 123.798,68; maio de 1998 está incompleto.

Limite de leitura: O ticket é calculado por pedido: a média dos valores dos itens mediria o item médio, não o pedido médio. Um mês sem pedidos teria valor zero e ticket indefinido (nulo).

14.3 Q03/P03 — Produtos sem vendas no período

Pergunta: Quais produtos não foram vendidos no período?

Cálculo: Começar pelo cadastro completo e verificar se existe pelo menos um item em pedido dentro da janela.

Implementação: SQL: NOT EXISTS e junções cadastrais. MongoDB: \$lookup com \$limit e filtro de array vazio.

Resultado: zero linhas nos dois bancos.

Interpretação: A consulta retorna zero linhas: todos os 77 produtos aparecem em alguma venda do recorte completo. Uma saída vazia responde validamente à pergunta.

Limite de leitura: O resultado vazio vale apenas para o recorte completo; em janelas menores podem surgir produtos sem venda. Os testes sintéticos confirmam que a consulta lista esses produtos quando existem.

14.4 Q04/P04 — Indicadores por funcionário

Pergunta: Como se distribuem os pedidos entre os funcionários?

Cálculo: Preservar o cadastro de funcionários, somar pedidos e valor de cada um e calcular ticket e participação.

Implementação: SQL: LEFT JOIN, agregação e janela global. MongoDB: \$lookup, \$group e \$setWindowFields.

Resultado: 9 linhas conferidas; todas exibidas.

funcionário (ID)	funcionário	pedidos	valor	ticket	participação (%)
1	Nancy Davolio	123	192.107,60	1.561,85	15,18
2	Andrew Fuller	96	166.537,76	1.734,77	13,16
3	Janet Leverling	127	202.812,84	1.596,95	16,02
4	Margaret Peacock	156	232.890,85	1.492,89	18,40
5	Steven Buchanan	42	68.792,28	1.637,91	5,43
6	Michael Suyama	67	73.913,13	1.103,18	5,84
7	Robert King	72	124.568,24	1.730,11	9,84
8	Laura Callahan	104	126.862,28	1.219,83	10,02
9	Anne Dodsworth	43	77.308,07	1.797,86	6,11

Interpretação: Margaret Peacock tem o maior valor registrado: 232.890,85, associado a 156 pedidos e 18,40% do total. A distribuição não controla território, carteira ou tempo de trabalho.

Limite de leitura: Os valores descrevem vendas registradas por funcionário. Sem informação de carteira, território ou tempo de atuação, a diferença entre eles não mede produtividade.

14.5 Q05/P05 — Estoque crítico e cobertura por categoria

Pergunta: Quais categorias têm produtos abaixo do ponto de reposição?

Cálculo: Comparar estoque com reorder_level, somar a diferença e verificar se estoque mais encomendado cobre o ponto.

Implementação: SQL: Comparação entre colunas, FILTER e array_agg. MongoDB: \$expr, \$set, \$group e \$push.

Resultado: 7 linhas conferidas; todas exibidas.

categoria (ID)	categoria	produtos	déficit	cobertura insuficiente	produtos (IDs)
1	Beverages	3	31	1	2, 43, 70
2	Condiments	2	28	0	3, 66
3	Confections	4	26	0	21, 48, 49, 68
4	Dairy Products	3	44	0	11, 31, 32
5	Grains/Cereals	2	17	0	56, 64
7	Produce	1	1	0	74
8	Seafood	3	29	1	30, 37, 45

Interpretação: Há 18 produtos abaixo do ponto, em sete categorias. Em dois deles, nem o estoque somado ao encomendado alcança o ponto. Um dos 18, Chang (ID 2), está descontinuado, o que mostra por que o sinal precisa ser conferido antes de uma compra.

Limite de leitura: O estoque é o estado cadastrado, sem reconstrução histórica. O indicador precisa ser conferido com o status do produto antes de virar recomendação de compra.

14.6 Q06/P06 — Clientes inativos e sem compras

Pergunta: Quais clientes estão sem compra há mais de seis meses?

Cálculo: Para cada cliente, obter a última compra até 06/05/1998. Selecionar data anterior a 06/11/1997 ou ausência de compras.

Implementação: SQL: LEFT JOIN, MAX e comparação de datas. MongoDB: \$lookup, \$dateDiff e tratamento de null.

Resultado: 7 linhas conferidas; todas exibidas.

cliente (código)	cliente	última compra	recência (dias)	situação
CENTC	Centro comercial Moctezuma	1996-07-18	657	inativo
FAMIA	Familia Arquibaldo	1997-10-31	187	inativo
FISSA	FISSA Fabrica Inter. Salchichas S.A.	—	—	sem compras
HUNGC	Hungry Coyote Import Store	1997-09-08	240	inativo
LAZYK	Lazy K Kountry Store	1997-05-22	349	inativo
MEREP	Mère Paillard	1997-10-30	188	inativo
PARIS	Paris spécialités	—	—	sem compras

Interpretação: São cinco clientes com compra anterior ao corte e dois sem compras registradas (FISSA e PARIS), situações que pedem abordagens diferentes. MEREP, nono maior cliente em valor e classe A na curva ABC, está entre os inativos: sua última compra foi em 30/10/1997.

Limite de leitura: A referência 06/05/1998 é o último pedido da base; a data atual tornaria todos os clientes inativos. Seis meses de calendário não equivalem necessariamente a 180 dias.

14.7 Q07/P07 — Top cinco produtos por categoria

Pergunta: Quais são os cinco produtos de maior valor em cada categoria?

Cálculo: Agregar por produto e categoria, ordenar por valor decrescente e ID crescente, numerar dentro de cada categoria e manter posições de 1 a 5.

Implementação: SQL: ROW_NUMBER com PARTITION BY. MongoDB: \$setWindowFields e soma cumulativa de 1.

Resultado: 40 linhas conferidas; exibido o primeiro colocado de cada categoria.

categoria (ID)	produto (ID)	valor	posição
1	38	141.396,74	1
2	63	16.701,10	1
3	62	47.234,97	1
4	59	71.155,70	1
5	56	42.593,06	1
6	29	80.368,67	1
7	51	41.819,65	1
8	18	29.171,88	1

Interpretação: O produto 38 (Côte de Blaye) lidera Beverages com 141.396,74, cerca de seis vezes o segundo colocado da categoria e 52,79% do valor de Beverages. Sozinho, ele responde por 11,17% do valor total do período, o que torna a categoria dependente de um único item.

Limite de leitura: O ranking é por valor, não por unidades vendidas. Com ROW_NUMBER, empates na quinta posição são resolvidos pelo menor ID; RANK poderia devolver mais de cinco linhas.

14.8 Q08/P08 — Valor mensal acumulado

Pergunta: Qual é o valor acumulado ao longo dos meses?

Cálculo: Construir a série mensal e somar os valores da primeira linha até a linha corrente.

Implementação: SQL: SUM OVER com ROWS UNBOUNDED PRECEDING. MongoDB: \$setWindowFields com janela até current.

Resultado: 23 linhas conferidas; todas exibidas.

mês	valor	acumulado
1996-07-01	27.861,90	27.861,90
1996-08-01	25.485,28	53.347,17
1996-09-01	26.381,40	79.728,57
1996-10-01	37.515,73	117.244,30
1996-11-01	45.600,05	162.844,34
1996-12-01	45.239,63	208.083,97
1997-01-01	61.258,07	269.342,04
1997-02-01	38.483,64	307.825,68
1997-03-01	38.547,22	346.372,90
1997-04-01	53.032,95	399.405,85
1997-05-01	53.781,29	453.187,14
1997-06-01	36.362,80	489.549,94
1997-07-01	51.020,86	540.570,80
1997-08-01	47.287,67	587.858,47
1997-09-01	55.629,24	643.487,71
1997-10-01	66.749,23	710.236,94
1997-11-01	43.533,81	753.770,75
1997-12-01	71.398,43	825.169,17
1998-01-01	94.222,11	919.391,28
1998-02-01	99.415,29	1.018.806,57
1998-03-01	104.854,16	1.123.660,73
1998-04-01	123.798,68	1.247.459,41
1998-05-01	18.333,63	1.265.793,04

Interpretação: O acumulado termina em 1.265.793,04, reconciliando com o total independente dos itens. O acumulado aumenta mesmo quando o ritmo mensal desacelera.

Limite de leitura: Um acumulado crescente não indica crescimento mensal: ele sobe sempre que o mês tem valor positivo. O total é a última linha, não a soma dos acumulados.

14.9 Q09/P09 — Variação mensal do valor

Pergunta: Quanto o valor mudou em relação ao mês anterior?

Cálculo: Obter o valor do mês anterior, subtrair do atual e dividir a diferença pelo anterior.

Implementação: SQL: LAG e NULLIF. MongoDB: \$shift e \$cond.

Resultado: 23 linhas conferidas; todas exibidas.

mês	valor	mês anterior	variação	variação (%)
1996-07-01	27.861,90	—	—	—
1996-08-01	25.485,28	27.861,90	-2.376,62	-8,53
1996-09-01	26.381,40	25.485,28	896,13	3,52
1996-10-01	37.515,73	26.381,40	11.134,33	42,21
1996-11-01	45.600,05	37.515,73	8.084,32	21,55
1996-12-01	45.239,63	45.600,05	-360,42	-0,79
1997-01-01	61.258,07	45.239,63	16.018,44	35,41
1997-02-01	38.483,64	61.258,07	-22.774,44	-37,18

mês	valor	mês anterior	variação	variação (%)
1997-03-01	38.547,22	38.483,64	63,59	0,17
1997-04-01	53.032,95	38.547,22	14.485,73	37,58
1997-05-01	53.781,29	53.032,95	748,34	1,41
1997-06-01	36.362,80	53.781,29	-17.418,49	-32,39
1997-07-01	51.020,86	36.362,80	14.658,06	40,31
1997-08-01	47.287,67	51.020,86	-3.733,19	-7,32
1997-09-01	55.629,24	47.287,67	8.341,57	17,64
1997-10-01	66.749,23	55.629,24	11.119,98	19,99
1997-11-01	43.533,81	66.749,23	-23.215,42	-34,78
1997-12-01	71.398,43	43.533,81	27.864,62	64,01
1998-01-01	94.222,11	71.398,43	22.823,68	31,97
1998-02-01	99.415,29	94.222,11	5.193,18	5,51
1998-03-01	104.854,16	99.415,29	5.438,87	5,47
1998-04-01	123.798,68	104.854,16	18.944,53	18,07
1998-05-01	18.333,63	123.798,68	-105.465,05	-85,19

Interpretação: Agosto de 1996 ficou 8,53% abaixo de julho. A maior alta ocorreu em dezembro de 1997 (+64,01%). A queda de 85,19% em maio de 1998 não indica retração: a base só registra pedidos até 06/05/1998.

Limite de leitura: Quando o mês anterior tem valor zero, o percentual fica indefinido. O calendário completo impede que um mês sem vendas seja pulado e que meses não consecutivos sejam comparados.

14.10 Q10/P10 — Curva ABC de clientes

Pergunta: Como os clientes se distribuem pela curva ABC de valor?

Cálculo: Somar valor por cliente, ordenar, calcular acumulado e aplicar os limites de 80% e 95% ao acumulado anterior.

Implementação: SQL: CTEs, janela cumulativa e CASE. MongoDB: \$lookup, duas janelas e \$switch.

Resultado: 91 linhas conferidas; exibidas as cinco primeiras na ordenação definida.

cliente (código)	cliente	pedidos	valor	acumulado	classe
QUICK	QUICK-Stop	28	110.277,31	110.277,31	A
ERNSH	Ernst Handel	30	104.874,98	215.152,28	A
SAVEA	Save-a-lot Markets	31	104.361,95	319.514,23	A
RATTC	Rattlesnake Canyon Grocery	18	51.097,80	370.612,03	A
HUNGO	Hungry Owl All-Night Grocers	19	49.979,91	420.591,94	A

Resumo das 91 linhas por classe, calculado a partir da mesma saída validada:

classe	clientes	valor	participação (%)
A	34	1.013.547,90	80,07
B	27	192.826,66	15,23
C	28	59.418,47	4,69
sem compras	2	0,00	0,00

Interpretação: A classificação produz 34 clientes A, 27 B, 28 C e dois sem compras. QUICK-Stop lidera com 110.277,31. Os grupos representam concentração de valor, não probabilidades futuras de compra.

Limite de leitura: A classificação depende da concentração do valor, não da quantidade de clientes; por isso NTILE não seria equivalente. O cliente que cruza um limite permanece na classe que estava sendo completada.

14.11 Q11/P11 — RFM descritivo dos clientes

Pergunta: Qual é o perfil de recência, frequência e valor de cada cliente?

Cálculo: Recência são dias desde a última compra; frequência é quantidade de pedidos; valor é a soma após descontos. Aplicar faixas fixas documentadas.

Implementação: SQL: CTE e CASE. MongoDB: \$lookup, \$dateDiff e condições.

Resultado: 91 linhas conferidas; exibidas as cinco primeiras na ordenação definida.

cliente (código)	recência (dias)	frequência	valor	R
ALFKI	27	6	4.273,00	3
ANATR	63	4	1.402,95	2
ANTON	98	7	7.023,98	1
AROUT	26	13	13.390,65	3
BERGS	63	18	24.927,58	2

cliente (código)	F	M
ALFKI	2	1
ANATR	1	1
ANTON	2	2
AROUT	3	3
BERGS	3	3

Interpretação: ALFKI tem recência de 27 dias, seis pedidos e valor de 4.273,00, resultando em R=3, F=2 e M=1. Os três números explicam aspectos diferentes do relacionamento.

Limite de leitura: As faixas são convenções didáticas, não um modelo estatístico. Clientes sem compras recebem escores zero e recência nula, sem data inventada.

14.12 Q12/P12 — Produtos comprados juntos

Pergunta: Quais produtos aparecem juntos no mesmo pedido?

Cálculo: Formar combinações de dois IDs diferentes no pedido, mantendo apenas A menor que B, contar pedidos por par e dividir pelo total de pedidos.

Implementação: SQL: Autojunção de itens e agregação. MongoDB: Expansão de arrays, \$match e \$group.

Resultado: 1535 linhas conferidas; exibidas as cinco primeiras na ordenação definida.

produto A	produto B	pedidos	pedidos no período	suporte (%)
21	61	8	830	0,96
16	31	7	830	0,84
16	60	6	830	0,72
16	62	6	830	0,72
30	54	6	830	0,72

Interpretação: Foram observados 1.535 pares distintos. O par 21–61 lidera com oito pedidos, suporte de aproximadamente 0,96%. A frequência é pequena e não comprova que uma promoção conjunta funcionaria.

Limite de leitura: Cada par é contado uma vez por pedido, com o menor ID primeiro. O denominador inclui pedidos de um único produto. Frequência conjunta não demonstra efeito causal.

14.13 Q13/P13 — Intervalo até envio por transportadora

Pergunta: Qual é o intervalo até o envio registrado por transportadora?

Cálculo: Subtrair a data do pedido da data de envio onde ela existe; contar ausências separadamente e preservar as transportadoras sem pedidos.

Implementação: SQL: Diferença de datas, AVG e FILTER. MongoDB: \$lookup, \$dateDiff e contagens condicionais.

Resultado: 6 linhas conferidas; todas exibidas.

transportadora (ID)	transportadora	pedidos	com data de envio	sem data de envio
1	Speedy Express	249	245	4
2	United Package	326	315	11
3	Federal Shipping	255	249	6
4	Alliance Shippers	0	0	0
5	UPS	0	0	0
6	DHL	0	0	0

transportadora (ID)	soma de dias	média (dias)	após data requerida
1	2100	8,57	12
2	2909	9,23	16
3	1861	7,47	9
4	0	—	0
5	0	—	0
6	0	—	0

Interpretação: Speedy Express apresenta média de 8,57 dias sobre 245 envios registrados, com quatro datas ausentes em 249 pedidos. No conjunto, há 21 datas ausentes e 37 envios registrados depois da data requerida. As três transportadoras adicionais do cadastro não têm pedidos.

Limite de leitura: O intervalo vai até o envio registrado e não mede a entrega ao cliente. Datas ausentes não entram na média como zero nem provam que o pedido não foi enviado.

14.14 Q14/P14 — Descontos sobre o valor dos pedidos

Pergunta: Quanto os descontos reduzem o valor bruto dos itens?

Cálculo: Somar preço vezes quantidade, somar desconto monetário e subtrair para obter valor após descontos; dividir desconto pelo bruto para a taxa ponderada.

Implementação: SQL: Expressões numeric, SUM e razão de somas. MongoDB: Expressões Decimal128, \$group e \$divide.

Resultado: 8 linhas conferidas; todas exibidas.

categoria (ID)	valor bruto	desconto	valor	desconto (%)
1	286.526,95	18.658,77	267.868,18	6,51
2	113.694,75	7.647,67	106.047,09	6,73
3	177.099,10	9.741,88	167.357,23	5,50
4	251.330,50	16.823,22	234.507,29	6,69
5	100.726,80	4.982,21	95.744,59	4,95
6	178.188,80	15.166,44	163.022,36	8,51
7	105.268,60	5.284,02	99.984,58	5,02
8	141.623,09	10.361,35	131.261,74	7,32

Interpretação: Em Beverages, o desconto soma 18.658,77 sobre bruto de 286.526,95, aproximadamente 6,51%. O cálculo mede redução registrada, sem estimar efeito do desconto sobre a demanda.

Limite de leitura: A taxa é ponderada pelo valor bruto, diferente da média simples das taxas dos itens. Sem custos, o desconto não pode ser convertido em perda de lucro.

14.15 Q15/P15 — Vendas próprias e da equipe

Pergunta: Quanto foi registrado por cada funcionário e sua equipe?

Cálculo: Percorrer os subordinados, incluir o próprio responsável e somar as vendas de cada membro uma vez dentro da equipe.

Implementação: SQL: WITH RECURSIVE e pré-agregação. MongoDB: \$graphLookup, \$setUnion e \$lookup.

Resultado: 9 linhas conferidas; todas exibidas.

gestor (ID)	membros	valor próprio	pedidos da equipe	valor da equipe
1	1	192.107,60	123	192.107,60
2	9	166.537,76	830	1.265.793,04
3	1	202.812,84	127	202.812,84
4	1	232.890,85	156	232.890,85
5	4	68.792,28	224	344.581,71
6	1	73.913,13	67	73.913,13
7	1	124.568,24	72	124.568,24
8	1	126.862,28	104	126.862,28
9	1	77.308,07	43	77.308,07

Interpretação: Andrew Fuller, gestor 2, reúne nove membros e os 830 pedidos da base. Seu valor próprio é 166.537,76; o valor de sua equipe é o total geral.

Limite de leitura: As equipes se sobrepõem na hierarquia; somar as linhas de todos os gestores contaria pedidos mais de uma vez. A equipe hierárquica não corresponde à divisão por território.

14.16 Q16/P16 — Distribuição trimestral por ano e categoria

Pergunta: Como o valor se distribui por ano, trimestre e categoria?

Cálculo: Produzir totais trimestrais e sua decomposição por categoria; marcar trimestres cuja cobertura ultrapassa as datas disponíveis.

Implementação: SQL: GROUPING SETS e GROUPING. MongoDB: \$facet, \$concatArrays e agregação temporal.

Resultado: 72 linhas conferidas; exibidos os oito totais trimestrais.

ano	trimestre	categoria (ID)	valor	nível	parcial
1996	3	—	79.728,57	total	sim
1996	4	—	128.355,40	total	não
1997	1	—	138.288,93	total	não
1997	2	—	143.177,05	total	não
1997	3	—	153.937,77	total	não
1997	4	—	181.681,46	total	não
1998	1	—	298.491,55	total	não
1998	2	—	142.132,31	total	sim

Interpretação: São 72 linhas, com oito totais trimestrais e 64 categorias por trimestre. O terceiro trimestre de 1996 e o segundo de 1998 são parciais na cobertura do conjunto.

Limite de leitura: Totais e categorias estão em níveis diferentes e não devem ser somados juntos. Com menos de dois anos completos, a série não permite afirmar sazonalidade recorrente.

15. Recursos documentais e MapReduce

15.1 Arrays, redução e múltiplas saídas

A primeira demonstração seleciona pedidos que tenham o produto 11, quantidade de pelo menos dez e desconto positivo no **mesmo item**, usando `$elemMatch`. `$filter` apresenta somente os itens correspondentes. Condições independentes sobre um array poderiam ser satisfeitas por itens diferentes, produzindo uma interpretação incorreta.

A segunda demonstração mantém um documento por pedido e calcula seu valor por `$reduce`, enquanto `$map` extrai os nomes dos produtos. Para o pedido 10248, os itens totalizam 440,00. Não é necessário expandir os itens em várias linhas para responder a essa pergunta.

A terceira usa `$facet` para devolver volume, presença de data de envio e distribuição da quantidade de itens dos pedidos de 1997. Cada faceta responde a uma pergunta sobre o mesmo recorte. A utilidade do formato não implica vantagem de desempenho em qualquer cenário.

Com `$elemMatch`, 11 pedidos têm um item que atende às três condições ao mesmo tempo. Com as mesmas condições escritas separadamente sobre o array, a contagem sobe para 16: nos 5 pedidos a mais, as condições são satisfeitas por itens diferentes.

O pedido 10248 contém Queso Cabrales, Singaporean Hokkien Fried Mee, Mozzarella di Giovanni; o `$reduce` somou 440,00 sem expandir os itens. O painel de 1997 devolveu, em um único documento, 408 pedidos, a situação da data de envio e a distribuição por quantidade de itens:

Faceta	Resultado
volume	408 pedidos
envio	com data: 408
tamanho	1 item: 62; 2 itens: 139; 3 itens: 127; 4 itens: 63; 5 itens: 16; 6 itens: 1

15.2 MapReduce e equivalente moderno

A análise MapReduce soma as quantidades vendidas por produto. A função `map` emite o ID e a quantidade de cada item; `reduce` soma os valores de cada produto. O resultado foi comparado ao pipeline com `$unwind` e `$group`: **77 produtos, 51.317 unidades e igualdade de todos os totais**. A saída foi retornada em memória, sem criar coleção de resultados.

Essa demonstração usa quantidades inteiras, evitando conversões monetárias para números JavaScript. MapReduce está depreciado desde MongoDB 5.0; a documentação recomenda aggregation pipelines para novos desenvolvimentos [14]. O recurso foi executado para atender à atividade da disciplina e contextualizado como abordagem legada.

O código completo das três demonstrações e do MapReduce, incluindo sua comparação automática com o pipeline, está no Apêndice B.

16. Comparação de sintaxe e complexidade

SQL descreve o resultado desejado a partir de relações entre tabelas. Um pipeline descreve uma sequência de transformações aplicadas aos documentos. Nos dois casos há otimizador, e a ordem escrita não determina todos os detalhes da execução. A equivalência depende das regras de negócio e da granularidade, não apenas dos operadores. A tabela a seguir resume os recursos de cada par; o número de linhas de SQL (sem comentários) e de estágios do pipeline (no nível principal, sem contar os estágios internos de \$lookup e \$facet) é apenas um indicador auxiliar de tamanho, não de dificuldade.

Par	SQL: recursos	Linhas SQL	MongoDB: recursos	Estágios
01	JOIN, GROUP BY e SUM OVER	8	\$unwind, \$group e \$setWindowFields	6
02	CTEs, generate_series e agregação em dois níveis	10	\$map, \$group, \$facet e calendário por \$dateAdd	8
03	NOT EXISTS e junções cadastrais	7	\$lookup com \$limit e filtro de array vazio	4
04	LEFT JOIN, agregação e janela global	10	\$lookup, \$group e \$setWindowFields	5
05	Comparação entre colunas, FILTER e array_agg	7	\$expr, \$set, \$group e \$push	6
06	LEFT JOIN, MAX e comparação de datas	9	\$lookup, \$dateDiff e tratamento de null	5
07	ROW_NUMBER com PARTITION BY	7	\$setWindowFields e soma cumulativa de 1	7
08	SUM OVER com ROWS UNBOUNDED PRECEDING	10	\$setWindowFields com janela até current	9
09	LAG e NULLIF	13	\$shift e \$cond	10
10	CTEs, janela cumulativa e CASE	13	\$lookup, duas janelas e \$switch	5
11	CTE e CASE	11	\$lookup, \$dateDiff e condições	5
12	Autojunção de itens e agregação	8	Expansão de arrays, \$match e \$group	6
13	Diferença de datas, AVG e FILTER	7	\$lookup, \$dateDiff e contagens condicionais	4
14	Expressões numeric, SUM e razão de somas	4	Expressões Decimal128, \$group e \$divide	6
15	WITH RECURSIVE e pré-agregação	15	\$graphLookup, \$setUnion e \$lookup	5
16	GROUPING SETS e GROUPING	11	\$facet, \$concatArrays e agregação temporal	11

16.1 Ranking, ABC e RFM

No ranking, SQL agrega por produto e usa ROW_NUMBER particionado por categoria. MongoDB agrega os itens e usa uma janela com soma cumulativa de 1. Essa escolha preserva a ordenação por valor e ID: no MongoDB 7, \$documentNumber exige sortBy com um único campo [17]. O resultado final mantém as mesmas 40 linhas nos dois bancos.

Na curva ABC, a etapa decisiva é acumular o **valor** em ordem decrescente. Dividir clientes em grupos de tamanho igual não responderia à mesma pergunta. A classificação usa o acumulado anterior ao cliente para definir a classe; os dois clientes sem compras ficam identificados separadamente. Em RFM, os dois códigos usam as mesmas faixas fixas e preservam os valores brutos para permitir interpretação além dos escores.

16.2 Produtos comprados juntos

SQL relaciona order_items consigo mesma pela chave do pedido e impõe produto_a < produto_b. MongoDB projeta os IDs dos itens, expande as duas listas e aplica a mesma condição. Ela elimina o produto pareado

consigo próprio e as duplicações A–B/B–A. A quantidade de pares intermediários cresce com o tamanho do pedido, tornando a análise sensível à expansão dos arrays.

O denominador do suporte é a quantidade de pedidos do período, inclusive pedidos que não geram pares. O par mais frequente aparece em apenas oito de 830 pedidos. Frequência conjunta não comprova efeito causal nem eficácia de uma promoção.

16.3 Hierarquia e equipe

A CTE recursiva de PostgreSQL percorre funcionários subordinados. O \$graphLookup faz o percurso equivalente pelos documentos de employees. Agregar os pedidos por funcionário antes de combinar com a equipe reduz a quantidade de registros intermediários. A cópia SQL impede repetição no caminho; a hierarquia original foi conferida sem ciclos.

Cada equipe inclui seu responsável. Equipes de gestores diferentes podem se sobrepor: o total do gestor máximo contém as vendas de subordinados que também aparecem em outras linhas. Somar a coluna de todas as equipes contaria vendas mais de uma vez.

16.4 Efeito da modelagem

O documento de pedido já reúne itens, nomes de produtos e categorias; isso simplifica algumas leituras. A cópia dos nomes exige manutenção quando o cadastro muda. Consultas iniciadas por customers para incluir clientes sem pedidos usam \$lookup; consultas de hierarquia usam relações entre documentos. A incorporação não elimina toda necessidade de junção.

No SQL, as chaves e restrições explicitam a integridade entre tabelas. MongoDB usa validadores para a estrutura de cada documento e verificações separadas para referências e cópias. Legibilidade, consistência e manutenção devem ser avaliadas junto com o tempo observado, sem assumir um vencedor por paradigma.

17. Análise de performance e otimização

17.1 Método e ambiente

Os 16 pares foram medidos na mesma máquina, sobre os dados validados, com conexões persistentes dos drivers psycopg (PostgreSQL) e PyMongo (MongoDB). Cada medição inclui o envio da consulta, a execução, a transferência e o consumo de **todas** as linhas ou documentos pelo cliente. Abertura de conexão, inicialização de processos, impressão e gravação de arquivos ficaram fora do trecho cronometrado.

O cenário é de **cache aquecido**: cinco execuções de aquecimento e vinte medições por banco e consulta. A ordem entre PostgreSQL e MongoDB alterna a cada rodada. A mediana resume o valor central, e o intervalo entre o primeiro e o terceiro quartil (Q1–Q3) indica a dispersão. Não foram feitos teste de significância estatística nem teste de carga concorrente.

O ambiente executou PostgreSQL 16.15 e MongoDB 7.0.40 em containers Docker, no WSL2, com processador Intel(R) Core(TM) i7-14700HX (28 processadores lógicos visíveis) e 7,8 GB de memória visível no Linux. Os containers não tinham limite de CPU ou memória e compartilharam os recursos da máquina. Configuração relevante do PostgreSQL: `effective_cache_size = 4 GB`, `jit = on`, `max_parallel_workers_per_gather = 2`, `shared_buffers = 128 MB`, `work_mem = 4 MB`.

Os planos de execução foram coletados à parte, com EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON) e `explain("executionStats")` [15, 16], para não interferir nos tempos. Custos estimados e tempos internos de um servidor não são comparáveis diretamente com os do outro; a comparação de tempo usa apenas o cronômetro externo comum.

17.2 Tempos do catálogo

Par	PostgreSQL: mediana (ms)	PostgreSQL: Q1–Q3 (ms)	MongoDB: mediana (ms)	MongoDB: Q1–Q3 (ms)
01	1,011	0,991–1,039	2,281	2,222–2,304
02	1,391	1,321–1,421	2,475	2,319–2,651
03	0,503	0,482–0,533	2,846	2,774–2,967
04	1,340	1,306–1,420	3,916	3,748–3,966
05	0,218	0,213–0,231	0,502	0,491–0,545
06	0,426	0,384–0,469	6,312	5,879–6,645
07	1,162	1,130–1,199	2,711	2,629–2,787
08	1,380	1,326–1,414	2,475	2,396–2,573
09	1,389	1,342–1,455	2,520	2,416–2,653
10	1,594	1,560–1,625	6,597	6,473–6,842
11	1,504	1,457–1,591	6,771	6,576–7,064
12	2,872	2,800–2,960	8,242	7,925–8,691
13	0,375	0,356–0,396	2,411	2,256–2,462
14	1,441	1,419–1,483	3,644	3,561–3,739
15	1,258	1,229–1,292	5,706	5,605–5,975
16	1,480	1,429–1,513	4,443	4,298–4,567

O PostgreSQL apresentou menor mediana nos 16 pares nesta execução; em todos eles, os intervalos Q1–Q3 dos dois bancos não se sobrepõem. O resultado vale para estes códigos, drivers, volume, ambiente e índices. Não demonstra superioridade geral nem indica o comportamento com milhões de pedidos.

Dois fatores de configuração ajudam a interpretar a diferença. Primeiro, os índices não são simétricos: o PostgreSQL tem índices em `orders(customer_id)`, `orders(employee_id)`, `orders(order_date)` e `order_items(product_id)`, enquanto o MongoDB tem `cliente_data`, `produto_no_pedido` e `categoria_produto`. Sem índice em `employee_id` e `shipper_id`, cada \$lookup de P04 e P15 examinou os 830 pedidos para cada um dos 9 funcionários (7.470 documentos), e P13 examinou 4.980 documentos para as 6 transportadoras. Segundo, nos pares que partem de cadastros (clientes, funcionários, transportadoras), o pipeline executa uma subconsulta

por documento, enquanto o PostgreSQL resolve a junção de uma só vez. Criar índices equivalentes no MongoDB é um teste pendente, não um resultado deste relatório.

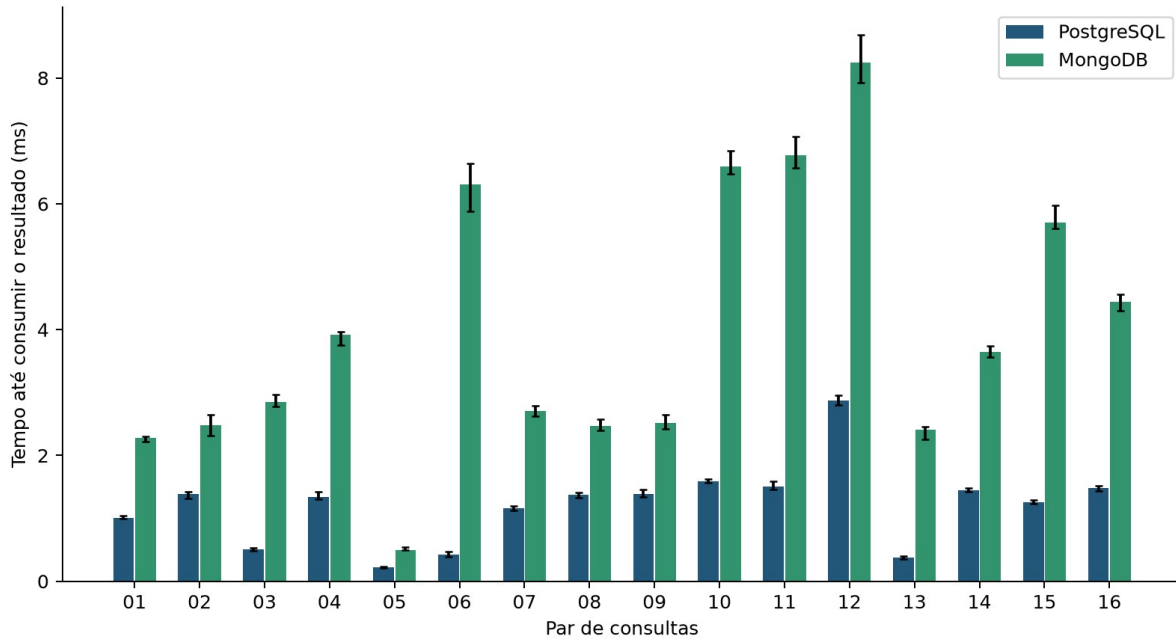


Figura 4 — Medianas das 20 medições por par, em milissegundos; as barras de erro indicam Q1 e Q3.

17.3 Estudos controlados

Os quatro estudos usam cópias isoladas dos dados. Nas cópias PostgreSQL, foram mantidos apenas os índices de PK e UNIQUE; nas cópias MongoDB, apenas o índice de `_id`. Para separar o efeito de um índice sem criá-lo ou removê-lo durante as medições, foram mantidas três cópias: sem índices adicionais, com índice em `order_date` e com índice composto (`customer_id`, `order_date`). Os bancos originais não receberam esses índices, e as estatísticas das cópias PostgreSQL foram atualizadas antes das medições.

Em cada estudo, todas as versões foram executadas na mesma sequência de rodadas. A ordem das versões muda a cada rodada, e a ordem dos bancos alterna. Assim, uma variação do ambiente ao longo do tempo não favorece sistematicamente a versão medida primeiro ou por último. Antes das medições, os resultados de todas as versões foram comparados e eram idênticos.

A diferença entre duas versões é considerada consistente nesta execução quando os intervalos Q1–Q3 não se sobrepõem. Isso é uma regra descritiva: não substitui repetição independente nem teste estatístico.

C01 — Filtro mensal

A versão original obtém o mês com `to_char` em cada pedido e compara o texto; no MongoDB, cria o campo com `$dateToString` antes do `$match`. A reescrita filtra o intervalo de datas diretamente. A terceira versão repete a reescrita na cópia com índice em `order_date`. As três devolvem o mesmo resultado para janeiro de 1997.

Versão	PostgreSQL: mediana (Q1–Q3), ms	MongoDB: mediana (Q1–Q3), ms
original	0,470 (0,452–0,492)	1,007 (0,958–1,062)
reescrita	0,364 (0,347–0,385)	0,684 (0,676–0,731)
com_indice	0,322 (0,310–0,347)	0,507 (0,488–0,526)

Efeito da reescrita do filtro (original → reescrita): no PostgreSQL, redução de 22,4% na mediana, sem sobreposição dos intervalos Q1–Q3; no MongoDB, redução de 32,1% na mediana, sem sobreposição dos intervalos Q1–Q3.

Efeito do índice por data (reescrita → com_indice): no PostgreSQL, redução de 11,5% na mediana, sem sobreposição dos intervalos Q1–Q3; no MongoDB, redução de 25,9% na mediana, sem sobreposição dos intervalos Q1–Q3.

Resumo dos planos de execução de C01:

Versão testada	Banco de dados	Operações do plano	Índice usado	Trabalho observado
original	PostgreSQL	Aggregate, Sort, Subquery Scan, Hash Join, Seq Scan, Hash	nenhum	797 linhas descartadas por filtro; 29 páginas lidas do cache
original	MongoDB	PROJECTION_SIMPLE, COLLSCAN	nenhum	830 documentos e 0 chaves examinados
reescrita	PostgreSQL	Aggregate, Sort, Subquery Scan, Hash Join, Seq Scan, Hash	nenhum	797 linhas descartadas por filtro; 29 páginas lidas do cache
reescrita	MongoDB	PROJECTION_SIMPLE, COLLSCAN	nenhum	830 documentos e 0 chaves examinados
com_indice	PostgreSQL	Aggregate, Sort, Subquery Scan, Hash Join, Seq Scan, Hash, Index Scan	orders_data	0 linhas descartadas por filtro; 18 páginas lidas do cache
com_indice	MongoDB	PROJECTION_SIMPLE, FETCH, IXSCAN	orders_data	33 documentos e 33 chaves examinados

C02 — Cliente e período

A mesma consulta — pedidos de VINET em 1996, ordenados por data e ID — foi executada na cópia sem índices adicionais e na cópia com índice composto (customer_id, order_date). O índice já existente no MongoDB original não foi usado como prova: o efeito foi medido nas cópias.

Versão	PostgreSQL: mediana (Q1–Q3), ms	MongoDB: mediana (Q1–Q3), ms
sem_composto	0,199 (0,183–0,206)	0,644 (0,632–0,664)
com_composto	0,173 (0,160–0,192)	0,441 (0,429–0,477)

Efeito do índice composto (sem_composto → com_composto): no PostgreSQL, redução de 13,1% na mediana, com sobreposição dos intervalos Q1–Q3, portanto inconclusiva; no MongoDB, redução de 31,5% na mediana, sem sobreposição dos intervalos Q1–Q3.

Resumo dos planos de execução de C02:

Versão testada	Banco de dados	Operações do plano	Índice usado	Trabalho observado
sem_composto	PostgreSQL	Sort, Seq Scan	nenhum	827 linhas descartadas por filtro; 15 páginas lidas do cache
sem_composto	MongoDB	SORT, PROJECTION_SIMPLE, COLLSCAN	nenhum	830 documentos e 0 chaves examinados
com_composto	PostgreSQL	Incremental Sort, Index Scan	orders_cliente_data	0 linhas descartadas por filtro; 3 páginas lidas do cache
com_composto	MongoDB	SORT, PROJECTION_SIMPLE, FETCH, IXSCAN	orders_cliente_data	3 documentos e 3 chaves examinados

C03 — Pares de produtos

A versão original em SQL foi escrita com uma CTE MATERIALIZED que gera todos os pares de itens de cada pedido antes de aplicar $A < B$. Sem essa instrução, o PostgreSQL 16 incorporaria a CTE e aplicaria a condição durante a junção, como faz a reescrita (Q12). O estudo mede, portanto, o custo de materializar os pares intermediários, não uma limitação do otimizador. No MongoDB, a versão original mantém todos os campos do pedido e só descarta os pares $A \geq B$ depois do \$group; a reescrita (P12) projeta somente os IDs e filtra antes de agrupar. Nenhum índice mudou.

Versão	PostgreSQL: mediana (Q1–Q3), ms	MongoDB: mediana (Q1–Q3), ms
original	3,353 (3,293–3,443)	8,300 (8,062–8,710)
reescrita	2,958 (2,871–3,059)	8,137 (7,921–8,592)

Efeito da reescrita (original → reescrita): no PostgreSQL, redução de 11,8% na mediana, sem sobreposição dos intervalos Q1–Q3; no MongoDB, redução de 2,0% na mediana, com sobreposição dos intervalos Q1–Q3, portanto inconclusiva.

Resumo dos planos de execução de C03:

Versão testada	Banco de dados	Operações do plano	Índice usado	Trabalho observado
original	PostgreSQL	Sort, Hash Join, Seq Scan, Hash, Nested Loop, Aggregate, CTE Scan	nenhum	4.607 linhas descartadas por filtro; 58 páginas lidas do cache
original	MongoDB	PROJECTION_DEFAULT, COLLSCAN	nenhum	830 documentos e 0 chaves examinados
reescrita	PostgreSQL	Sort, Nested Loop, Aggregate, Seq Scan, Hash Join, Hash	nenhum	0 linhas descartadas por filtro; 58 páginas lidas do cache
reescrita	MongoDB	PROJECTION_DEFAULT, COLLSCAN	nenhum	830 documentos e 0 chaves examinados

C04 — Vendas da equipe

A versão original combina a hierarquia com cada pedido e só depois agrega; a reescrita (Q15) agrega os pedidos por funcionário antes da combinação. No MongoDB, a versão original traz os pedidos completos no \$lookup e calcula depois; a reescrita (P15) agrupa dentro do \$lookup. Nenhum índice mudou.

Versão	PostgreSQL: mediana (Q1–Q3), ms	MongoDB: mediana (Q1–Q3), ms
original	1,558 (1,470–1,630)	6,440 (6,095–6,794)
reescrita	1,301 (1,261–1,355)	5,856 (5,700–6,007)

Efeito da reescrita (original → reescrita): no PostgreSQL, redução de 16,5% na mediana, sem sobreposição dos intervalos Q1–Q3; no MongoDB, redução de 9,1% na mediana, sem sobreposição dos intervalos Q1–Q3.

Resumo dos planos de execução de C04:

Versão testada	Banco de dados	Operações do plano	Índice usado	Trabalho observado
original	PostgreSQL	Aggregate, Recursive Union, Seq Scan, Hash Join, WorkTable Scan, Hash, Sort, CTE Scan	nenhum	0 linhas descartadas por filtro; 31 páginas lidas do cache
original	MongoDB	PROJECTION_SIMPLE, COLLSCAN	nenhum	9 documentos e 0 chaves examinados; \$lookup: 7.470 documentos
reescrita	PostgreSQL	Aggregate, Recursive Union, Seq Scan, Hash Join, WorkTable Scan, Hash, Sort, CTE Scan	nenhum	0 linhas descartadas por filtro; 31 páginas lidas do cache
reescrita	MongoDB	PROJECTION_SIMPLE, COLLSCAN	nenhum	9 documentos e 0 chaves examinados; \$lookup: 7.470 documentos

17.4 Leitura dos planos e limitações

Seq Scan (PostgreSQL) e COLLSCAN (MongoDB) indicam leitura de todas as linhas ou documentos; Index Scan e IXSCAN indicam acesso por índice. No MongoDB, FETCH indica a busca do documento depois da leitura do índice. Em C01, a versão original e a reescrita do PostgreSQL fazem a mesma varredura sequencial, que descarta 797 dos 830 pedidos. A diferença está no custo de converter e formatar a data com `to_char` em cada linha e na estimativa do otimizador: com `to_char` ele previu 4 pedidos (foram 33) e escolheu uma agregação com ordenação adicional; com o intervalo, previu 32 e usou agregação por hash. O índice só passa a ser usado na terceira versão. Em C02, o índice composto reduz a leitura a três pedidos nos dois bancos. Em C03 e C04, o objetivo é diminuir os resultados intermediários, não passar a usar índice.

Nos quadros, as linhas examinadas do PostgreSQL somam as linhas descartadas por filtros em todos os nós do plano. No MongoDB, as métricas do cursor inicial e as de cada \$lookup são registradas separadamente. Buffers compartilhados indicam acessos a páginas em memória, não necessariamente leituras de disco.

Não foram forçados índices com hint nem desligadas as varreduras sequenciais. Os dados cabem em memória, as consultas são curtas e o tempo medido inclui a comunicação local e a conversão de tipos pelos drivers. O consumo completo pesa mais em Q12/P12, que retorna 1.535 linhas. O estudo não mede concorrência, custo de escrita, sincronização entre os bancos nem escalabilidade.

18. Conclusões e reprodução

A Entrega 3 implementou 16 análises equivalentes, quatro views e três procedures, além de recursos documentais e MapReduce. A migração permaneceu consistente e todos os pares foram conferidos, incluindo valores, nulos, duplicidades e ordenação. A revisão padronizou o cálculo monetário sem arredondamento intermediário e distinguiu envio registrado de entrega efetiva.

18.1 Conclusões de negócio

Beverages concentra 21,16% do valor dos pedidos. Pela regra declarada, 34 dos 89 clientes com compras formam a classe A, que completa os primeiros 80% do valor. Cinco clientes não compram desde antes do corte de inatividade e dois nunca compraram; entre os inativos está MEREP, cliente da classe A, o caso que mais justifica uma ação de relacionamento. Esses resultados indicam onde investigar, sem provar o retorno financeiro de uma ação.

A análise de estoque identifica 18 produtos abaixo do ponto de reposição, dos quais dois não seriam cobertos pelas quantidades encomendadas. É necessário revisar a validade do cadastro e os produtos descontinuados antes de transformar o indicador em recomendação de compra.

Os intervalos por transportadora medem dias até o envio registrado e preservam as 21 ausências de data. Os pares de produtos mais frequentes têm suporte inferior a 1%; a base sustenta uma descrição das combinações, mas não uma conclusão causal sobre venda conjunta.

18.2 Conclusões técnicas

O PostgreSQL apresentou menor mediana nos 16 pares nesta execução local. A conclusão é restrita às consultas, aos índices e às condições medidas; em particular, o MongoDB não tinha índice em `employee_id` nem em `shipper_id`. O MongoDB reuniu itens e atributos de leitura no documento do pedido, mas as análises que partem de cadastros sem movimento ou percorrem a hierarquia ainda exigiram `$lookup` e `$graphLookup`. A escolha de tecnologia deve considerar integridade, manutenção e padrão de acesso, além do tempo.

Nos quatro estudos, as versões foram medidas de forma intercalada, e os efeitos da reescrita e do índice foram separados. As reduções observadas valem para esta execução e este volume; quando os intervalos Q1–Q3 se sobrepõem, a diferença foi tratada como inconclusiva. Escalabilidade e concorrência não foram medidas.

18.3 Reprodução local

O projeto usa Docker Compose com PostgreSQL e MongoDB e os drivers do ambiente Python. Após a carga e conferência das Entregas 1 e 2, executar, na raiz do projeto:

```
# Criar somente os objetos analíticos novos.
docker compose exec -T postgres psql -U pi -d northwind \
  -v ON_ERROR_STOP=1 -f /sql/entrega03/00_views.sql
docker compose exec -T postgres psql -U pi -d northwind \
  -v ON_ERROR_STOP=1 -f /sql/entrega03/10_procedures.sql
uv run python etl/validar_analytics.py
(cd etl && uv run python testar_analytics.py)
uv run python bench/entrega03.py
```

As demonstrações documentais podem ser executadas com mongosh nos arquivos `mongo/entrega03/recursos_documentais.js` e `mapreduce.js`. Os scripts P01–P16 podem ser executados individualmente por mongosh; os Q01–Q16 podem ser abertos no cliente SQL ou passados ao psql. O README contém os comandos de geração dos documentos e a sequência de reprodução.

A Entrega 4 consolidará as análises, a recomendação de uso de cada tecnologia e a apresentação final.

Referências

- [1] IBM. **CRISP-DM Help Overview**. IBM SPSS Modeler. Orientação sobre as fases de compreensão do negócio e dos dados. Disponível em: <https://www.ibm.com/docs/en/spss-modeler/18.6.0?topic=dm-crisp-help-overview>. Acesso em: 5 set. 2026.
- [2] PTHOM. **Northwind sample database for PostgreSQL**. Repositório da adaptação utilizada neste projeto. Disponível em: https://github.com/pthom/northwind_psql. Acesso em: 5 set. 2026.
- [3] IPOG. **Livro Digital — Projeto Integrador**. Unidade 4: Projeto Área 03 — Banco de Dados, seções 4.1, 4.2 e 4.5.2 a 4.5.6. Material didático da disciplina.
- [4] POSTGRES SQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 16 Documentation: Numeric Types**. Disponível em: <https://www.postgresql.org/docs/16/datatype-numeric.html>. Acesso em: 5 set. 2026.
- [5] MONGODB. **Best Practices for Data Modeling in MongoDB**. Disponível em: <https://www.mongodb.com/docs/manual/data-modeling/best-practices/>. Acesso em: 5 set. 2026.
- [6] MONGODB. **Model Monetary Data**. Disponível em: <https://www.mongodb.com/docs/manual/tutorial/model-monetary-data/>. Acesso em: 5 set. 2026.
- [7] MONGODB. **Schema Validation**. Disponível em: <https://www.mongodb.com/docs/manual/core/schema-validation/>. Acesso em: 5 set. 2026.
- [8] MONGODB. **Production Considerations: Transactions**. Disponível em: <https://www.mongodb.com/docs/manual/core/transactions-production-consideration/>. Acesso em: 5 set. 2026.
- [9] MONGODB. **MongoDB Extended JSON (v2)**. Disponível em: <https://www.mongodb.com/docs/manual/reference/mongodb-extended-json/>. Acesso em: 10 set. 2026.
- [10] MONGODB. **Multikey Indexes**. Disponível em: <https://www.mongodb.com/docs/manual/core/indexes/index-types/index-multikey/>. Acesso em: 10 set. 2026.
- [11] MONGODB. **\$round (expression operator)**. Disponível em: <https://www.mongodb.com/docs/manual/reference/operator/aggregation/round/>. Acesso em: 10 set. 2026.
- [12] POSTGRES SQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 16: JSON Functions and Operators**. Disponível em: <https://www.postgresql.org/docs/16/functions-json.html>. Acesso em: 10 set. 2026.
- [13] POSTGRES SQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 16: User-Defined Procedures e Cursors**. <https://www.postgresql.org/docs/16/xproc.html> e <https://www.postgresql.org/docs/16/plpgsql-cursors.html>. Acesso em: 22 set. 2026.
- [14] MONGODB. **MongoDB 7.0: Map-Reduce**. <https://www.mongodb.com/docs/v7.0/core/map-reduce/>. Acesso em: 22 set. 2026.
- [15] POSTGRES SQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 16: EXPLAIN**. <https://www.postgresql.org/docs/16/sql-explain.html>. Acesso em: 22 set. 2026.
- [16] MONGODB. **MongoDB 7.0: Aggregation Pipeline Optimization**. <https://www.mongodb.com/docs/v7.0/core/aggregation-pipeline-optimization/>. Acesso em: 22 set. 2026.
- [17] MONGODB. **MongoDB 7.0: \$documentNumber**. <https://www.mongodb.com/docs/v7.0/reference/operator/aggregation/documentnumber/>. Acesso em: 22 set. 2026.

Apêndice A — Consultas SQL e pipelines completos

Os códigos abaixo são os mesmos arquivos executados na validação. Cada consulta SQL pode ser executada no PostgreSQL e cada pipeline, no mongosh; o trecho final de cada arquivo .js apenas imprime o resultado. Datas e regras seguem o contrato do capítulo 12.

A.1 Par Q01/P01

PostgreSQL:

```
-- Q01 – Valor e participação por categoria
-- Pergunta: Quais categorias concentram o valor dos pedidos?
-- Resultado: Uma linha por categoria com vendas: unidades, valor após descontos e fatia (%) do
valor total.
-- Intervalo [1996-07-01, 1998-06-01); RFM/inatividade: 1998-05-06.
WITH totais AS (
  SELECT category_id, sum(quantity) unidades, sum(valor) valor
  FROM nw_analytics.vw_item_valor WHERE order_date >= DATE '1996-07-01' AND order_date < DATE
'1998-06-01' GROUP BY category_id
)
SELECT t.category_id, c.category_name categoria, t.unidades, t.valor,
100*t.valor/nullif(sum(t.valor) OVER (), 0) participacao
FROM totais t JOIN nw.categories c USING(category_id)
ORDER BY t.category_id;
MongoDB:
```

```
// P01 – Valor e participação por categoria
// Pergunta: Quais categorias concentram o valor dos pedidos?
// Resultado: Uma linha por categoria com vendas: unidades, valor após descontos e fatia (%) do
valor total.
// Datas UTC; Decimal128; saída ordenada equivalente ao SQL.
var consulta = {
  collection: "orders",
  pipeline: [
    {
      $match: {
        order_date: { $gte: ISODate("1996-07-01T00:00:00Z"), $lt: ISODate("1998-06-01T00:00:00Z")}
      }
    },
    { $unwind: "$items" },
    {
      $group: {
        _id: "$items.product.category._id",
        categoria: { $first: "$items.product.category.name" },
        unidades: { $sum: "$items.quantity" },
        valor: {
          $sum: {
            $multiply: [
              "$items.unit_price",
              "$items.quantity",
              { $subtract: [Decimal128("1"), "$items.discount"]}
            ]
          }
        }
      }
    },
    {
      $setWindowFields: { output: { total: { $sum: "$valor", window: { documents: ["unbounded", "unbounded"] } } } }
    },
    {
      $project: {
        _id: 0,
        category_id: "$_id",
        categoria: 1,
        unidades: 1,
        valor: 1,
        participacao: {
          $cond: [
            { $eq: ["$total", 0] },
            null,
            { $divide: [{ $multiply: ["$valor", Decimal128("100")] }, "$total"] }
          ]
        }
      }
    },
    { $sort: { category_id: 1 } }
  ]
};
```

```
if (typeof ANALYTICS_EXPORT === 'undefined') {
  print(EJSON.stringify(db.getSiblingDB('northwind')
    .getCollection(consulta.collection).aggregate(consulta.pipeline).toArray(), null, 2));
}
```

A.2 Par Q02/P02

PostgreSQL:

```
-- Q02 – Pedidos e ticket médio mensal
-- Pergunta: Como variam o número de pedidos e o ticket médio mensal?
-- Resultado: Uma linha por mês, inclusive sem pedidos: quantidade de pedidos, valor e valor
médio por pedido.
-- Intervalo [1996-07-01, 1998-06-01]; RFM/inatividade: 1998-05-06.
WITH calendario AS (
  SELECT generate_series(DATE '1996-07-01', DATE '1998-06-01'-INTERVAL '1 month', INTERVAL '1
month')::date mes
), totais AS (
  SELECT date_trunc('month',order_date)::date mes, count(*) pedidos, sum(valor) valor
  FROM nw_analytics.vw_pedido_valor WHERE order_date >= DATE '1996-07-01' AND order_date < DATE
'1998-06-01' GROUP BY 1
), mensal AS (
  SELECT c.mes,coalesce(t.pedidos,0) pedidos,coalesce(t.valor,0) valor
  FROM calendario c LEFT JOIN totais t USING(mes)
)
SELECT mes,pedidos,valor,valor/nullif(pedidos,0) ticket FROM mensal ORDER BY mes;
```

MongoDB:

```
// P02 – Pedidos e ticket médio mensal
// Pergunta: Como variam o número de pedidos e o ticket médio mensal?
// Resultado: Uma linha por mês, inclusive sem pedidos: quantidade de pedidos, valor e valor
médio por pedido.
// Datas UTC; Decimal128; saída ordenada equivalente ao SQL.
var consulta = {
  collection: "orders",
  pipeline: [
    {
      $match: {
        order_date: { $gte: ISODate("1996-07-01T00:00:00Z"), $lt: ISODate("1998-06-01T00:00:00Z")}
      }
    },
    {
      $set: {
        valor: {
          $sum: {
            $map: {
              input: "$items",
              as: "i",
              in: {
                $multiply: [
                  "$$i.unit_price",
                  "$$i.quantity",
                  {$subtract: [Decimal128("1"), "$$i.discount"]}
                ]
              }
            }
          }
        }
      }
    },
    {
      $facet: {
        dados: [
          {
            $group: {
              _id: {$dateTrunc: {date: "$order_date", unit: "month", timezone: "UTC"}},
              pedidos: {$sum: 1},
              valor: {$sum: "$valor"}
            }
          }
        ]
      }
    },
    {
      $project: {
        linhas: {
          $map: {
            input: {
              $range: [
                0,
                {

```

```

        $dateDiff: {
          startDate: ISODate("1996-07-01T00:00:00Z"),
          endDate: ISODate("1998-06-01T00:00:00Z"),
          unit: "month"
        }
      }
    ]
  },
  as: "k",
  in: {
    $let: {
      vars: {
        m: {
          $dateAdd: {
            startDate: ISODate("1996-07-01T00:00:00Z"),
            unit: "month",
            amount: "$$k"
          }
        }
      },
      in: {
        $let: {
          vars: {
            r: {
              $arrayElemAt: [
                {
                  $filter: {input: "$dados", as: "d", cond: {$eq: ["$$d._id", "$$m"]}}
                },
                0
              ]
            }
          },
          in: {
            mes: "$$m",
            pedidos: {$ifNull: ["$$r.pedidos", 0]},
            valor: {$ifNull: ["$$r.valor", Decimal128("0")]}
          }
        }
      }
    }
  }
},
{$unwind: "$linhas"},
{$replaceWith: "$linhas"},
{
  $project: {
    _id: 0,
    mes: {$dateToString: {date: "$mes", format: "%Y-%m-%d", timezone: "UTC"}},
    pedidos: 1,
    valor: 1,
    ticket: {$cond: [{ $eq: ["$pedidos", 0]}, null, {$divide: ["$valor", "$pedidos"]}] }
  },
  {$sort: {mes: 1}}
}
];
if (typeof ANALYTICS_EXPORT === 'undefined') {
  print(EJSON.stringify(db.getSiblingDB('northwind')
    .getCollection(consulta.collection).aggregate(consulta.pipeline).toArray(), null, 2));
}

```

A.3 Par Q03/P03

PostgreSQL:

```

-- Q03 – Produtos sem vendas no período
-- Pergunta: Quais produtos não foram vendidos no período?
-- Resultado: Produtos do catálogo sem nenhum item vendido na janela; lista vazia significa que todos venderam.
-- Intervalo [1996-07-01, 1998-06-01]; RFM/inatividade: 1998-05-06.
SELECT p.product_id,p.product_name produto,c.category_name categoria,s.company_name fornecedor
FROM nw.products p JOIN nw.categories c USING(category_id)
JOIN nw.suppliers s USING(supplier_id)
WHERE NOT EXISTS (
  SELECT 1 FROM nw.order_items i JOIN nw.orders o USING(order_id)
  WHERE i.product_id=p.product_id AND order_date >= DATE '1996-07-01' AND order_date < DATE '1998-06-01'
) ORDER BY p.product_id;

```

MongoDB:

```
// P03 – Produtos sem vendas no período
// Pergunta: Quais produtos não foram vendidos no período?
// Resultado: Produtos do catálogo sem nenhum item vendido na janela; lista vazia significa que
// todos venderam.
// Datas UTC; Decimal128; saída ordenada equivalente ao SQL.
var consulta = {
  collection: "products",
  pipeline: [
    {
      $lookup: {
        from: "orders",
        localField: "_id",
        foreignField: "items.product._id",
        pipeline: [
          {
            $match: {
              order_date: {
                $gte: ISODate("1996-07-01T00:00:00Z"),
                $lt: ISODate("1998-06-01T00:00:00Z")
              }
            }
          },
          {
            $limit: 1,
            $project: { _id: 0, encontrado: { $literal: true } }
          }
        ],
        as: "vendas"
      }
    },
    {
      $match: { vendas: { $size: 0 } }
    },
    {
      $project: {
        _id: 0,
        product_id: "$_id",
        produto: "$product_name",
        categoria: "$category.name",
        fornecedor: "$supplier.name"
      }
    },
    {
      $sort: { product_id: 1 }
    }
  ]
};
if (typeof ANALYTICS_EXPORT === 'undefined') {
  print(EJSON.stringify(db.getSiblingDB('northwind')
    .getCollection(consulta.collection).aggregate(consulta.pipeline).toArray(), null, 2));
}
```

A.4 Par Q04/P04**PostgreSQL:**

```
-- Q04 – Indicadores por funcionário
-- Pergunta: Como se distribuem os pedidos entre os funcionários?
-- Resultado: Uma linha por funcionário, inclusive sem pedidos: pedidos, valor, valor médio por
-- pedido e fatia do total.
-- Intervalo [1996-07-01, 1998-06-01); RFM/inatividade: 1998-05-06.
WITH t AS (
  SELECT e.employee_id, e.first_name || ' ' || e.last_name funcionario,
         count(o.order_id) pedidos, coalesce(sum(o.valor), 0) valor
  FROM nw.employees e LEFT JOIN nw_analytics.vw_pedido_valor o
    ON o.employee_id = e.employee_id AND order_date >= DATE '1996-07-01' AND order_date < DATE '1998-06-01'
  GROUP BY e.employee_id
)
SELECT *, valor/nullif(pedidos, 0) ticket,
       100*valor/nullif(sum(valor) OVER (), 0) participacao
FROM t ORDER BY employee_id;
```

MongoDB:

```
// P04 – Indicadores por funcionário
// Pergunta: Como se distribuem os pedidos entre os funcionários?
// Resultado: Uma linha por funcionário, inclusive sem pedidos: pedidos, valor, valor médio por
// pedido e fatia do total.
// Datas UTC; Decimal128; saída ordenada equivalente ao SQL.
var consulta = {
  collection: "employees",
  pipeline: [
    {
      $lookup: {
        from: "orders",
```

```

        localField: "_id",
        foreignField: "employee_id",
        pipeline: [
            {
                $match: {
                    order_date: {
                        $gte: ISODate("1996-07-01T00:00:00Z"),
                        $lt: ISODate("1998-06-01T00:00:00Z")
                    }
                }
            },
            {
                $group: {
                    _id: null,
                    pedidos: {$sum: 1},
                    valor: {
                        $sum: {
                            $sum: {
                                $map: {
                                    input: "$items",
                                    as: "i",
                                    in: {
                                        $multiply: [
                                            "$$i.unit_price",
                                            "$$i.quantity",
                                            {$subtract: [Decimal128("1"), "$$i.discount"]}
                                        ]
                                    }
                                }
                            }
                        }
                    }
                }
            },
            {
                $as: "vendas"
            },
            {
                $set: {
                    pedidos: {$ifNull: [{ $first: "$vendas.pedidos"}, 0]},
                    valor: {$ifNull: [{ $first: "$vendas.valor"}, Decimal128("0")]}
                }
            },
            {
                $setWindowFields: {output: {total: {$sum: "$valor", window: {documents: ["unbounded", "unbounded"]}}}}
            },
            {
                $project: {
                    _id: 0,
                    employee_id: "$_id",
                    funcionario: {$concat: ["$first_name", " ", "$last_name"]},
                    pedidos: 1,
                    valor: 1,
                    ticket: {$cond: [{ $eq: ["$pedidos", 0]}, null, {$divide: ["$valor", "$pedidos"]}],},
                    participacao: {
                        $cond: [
                            { $eq: ["$total", 0]},
                            null,
                            {$divide: [{ $multiply: ["$valor", Decimal128("100")]}, "$total"]}
                        ]
                    }
                }
            },
            {$sort: {employee_id: 1}}
        ]
    };
    if (typeof ANALYTICS_EXPORT === 'undefined') {
        print(EJSON.stringify(db.getSiblingDB('northwind')
            .getCollection(consulta.collection).aggregate(consulta.pipeline).toArray(), null, 2));
    }
}

```

A.5 Par Q05/P05

PostgreSQL:

```

-- Q05 – Estoque crítico e cobertura por categoria
-- Pergunta: Quais categorias têm produtos abaixo do ponto de reposição?
-- Resultado: Categorias com produtos abaixo do ponto de reposição, déficit somado e quantos nem
com o encomendado se cobrem.

```

```
-- Intervalo [1996-07-01, 1998-06-01); RFM/inatividade: 1998-05-06.
SELECT p.category_id,c.category_name categoria,count(*) produtos,
       sum(p.reorder_level-p.units_in_stock) deficit,
       count(*) FILTER (WHERE p.units_in_stock+p.units_on_order<p.reorder_level)
       reposicao_insuficiente,
       array_agg(p.product_id ORDER BY p.product_id) produtos_ids
FROM nw.products p JOIN nw.categories c USING(category_id)
WHERE p.units_in_stock<p.reorder_level
GROUP BY p.category_id,c.category_name ORDER BY p.category_id;
MongoDB:
```

```
// P05 – Estoque crítico e cobertura por categoria
// Pergunta: Quais categorias têm produtos abaixo do ponto de reposição?
// Resultado: Categorias com produtos abaixo do ponto de reposição, déficit somado e quantos nem
// com o encomendado se cobrem.
// Datas UTC; Decimal128; saída ordenada equivalente ao SQL.
var consulta = {
  collection: "products",
  pipeline: [
    {
      $match: { $expr: { $lt: [ "$units_in_stock", "$reorder_level" ] } } },
    {
      $set: {
        deficit: { $subtract: [ "$reorder_level", "$units_in_stock" ] },
        insuficiente: {
          $cond: [
            { $lt: [ { $add: [ "$units_in_stock", "$units_on_order" ] }, "$reorder_level" ] },
            1,
            0
          ]
        }
      }
    },
    { $sort: { _id: 1 } },
    {
      $group: {
        _id: "$category._id",
        categoria: { $first: "$category.name" },
        produtos: { $sum: 1 },
        deficit: { $sum: "$deficit" },
        reposicao_insuficiente: { $sum: "$insuficiente" },
        produtos_ids: { $push: "$_id" }
      }
    },
    {
      $project: {
        _id: 0,
        category_id: "$_id",
        categoria: 1,
        produtos: 1,
        deficit: 1,
        reposicao_insuficiente: 1,
        produtos_ids: 1
      }
    },
    { $sort: { category_id: 1 } }
  ]
};
if (typeof ANALYTICS_EXPORT === 'undefined') {
  print(EJSON.stringify(db.getSiblingDB('northwind')
    .getCollection(consulta.collection).aggregate(consulta.pipeline).toArray(), null, 2));
}
```

A.6 Par Q06/P06

PostgreSQL:

```
-- Q06 – Clientes inativos e sem compras
-- Pergunta: Quais clientes estão sem compra há mais de seis meses?
-- Resultado: Clientes sem compra desde antes de 06/11/1997 (inativos) e clientes que nunca
-- compraram.
-- Intervalo [1996-07-01, 1998-06-01); RFM/inatividade: 1998-05-06.
WITH historico AS (
  SELECT c.customer_id,c.company_name cliente,max(o.order_date) ultima
  FROM nw.customers c LEFT JOIN nw.orders o ON o.customer_id=c.customer_id
  AND o.order_date<=DATE '1998-05-06' GROUP BY c.customer_id
)
SELECT *,DATE '1998-05-06'-ultima recencia,
       CASE WHEN ultima IS NULL THEN 'sem compras' ELSE 'inativo' END situacao
FROM historico WHERE ultima IS NULL OR ultima<DATE '1997-11-06'
ORDER BY customer_id;
```

MongoDB:

```
// P06 – Clientes inativos e sem compras
// Pergunta: Quais clientes estão sem compra há mais de seis meses?
// Resultado: Clientes sem compra desde antes de 06/11/1997 (inativos) e clientes que nunca
compraram.
// Datas UTC; Decimal128; saída ordenada equivalente ao SQL.
var consulta = {
  collection: "customers",
  pipeline: [
    {
      $lookup: {
        from: "orders",
        localField: "_id",
        foreignField: "customer_id",
        pipeline: [
          {
            $match: {order_date: {$lte: ISODate("1998-05-06T00:00:00Z")}},
            {
              $group: {
                _id: null,
                ultima: {$max: "$order_date"},
                frequencia: {$sum: 1},
                valor: {
                  $sum: {
                    $sum: {
                      $map: {
                        input: "$items",
                        as: "i",
                        in: {
                          $multiply: [
                            "$$i.unit_price",
                            "$$i.quantity",
                            {$subtract: [Decimal128("1"), "$$i.discount"]}
                          ]
                        }
                      }
                    }
                  }
                }
              }
            }
          ]
        }
      },
      as: "compras"
    },
    {
      $set: {
        ultima: {$ifNull: [{ $first: "$compras.ultima"}, null]},
        frequencia: {$ifNull: [{ $first: "$compras.frequencia"}, 0]},
        valor: {$ifNull: [{ $first: "$compras.valor"}, Decimal128("0")]}
      }
    },
    {
      $match: {$or: [{ultima: null}, {ultima: {$lt: ISODate("1997-11-06T00:00:00Z")}}]}
    },
    {
      $project: {
        _id: 0,
        customer_id: "$_id",
        cliente: "$company_name",
        ultima: {
          $dateToString: {date: "$ultima", format: "%Y-%m-%d", timezone: "UTC", onNull: null}
        },
        recencia: {
          $dateDiff: {startDate: "$ultima", endDate: ISODate("1998-05-06T00:00:00Z"), unit:
"day"}
        },
        situacao: {$cond: [{ $eq: ["$ultima", null]}, "sem compras", "inativo"]}
      }
    },
    {
      $sort: {customer_id: 1}
    }
  ]
};
if (typeof ANALYTICS_EXPORT === 'undefined') {
  print(EJSON.stringify(db.getSiblingDB('northwind')
    .getCollection(consulta.collection).aggregate(consulta.pipeline).toArray(), null, 2));
}
```

A.7 Par Q07/P07

PostgreSQL:

```
-- Q07 – Top cinco produtos por categoria
```

```
-- Pergunta: Quais são os cinco produtos de maior valor em cada categoria?
-- Resultado: Até cinco produtos de maior valor em cada categoria, com posição 1 a 5 e desempate
pelo menor ID.
-- Intervalo [1996-07-01, 1998-06-01]; RFM/inatividade: 1998-05-06.
WITH t AS (
  SELECT category_id,product_id,sum(valor) valor FROM nw_analytics.vw_item_valor
  WHERE order_date >= DATE '1996-07-01' AND order_date < DATE '1998-06-01' GROUP BY
  category_id,product_id
), ranking AS (
  SELECT *,row_number() OVER (PARTITION BY category_id ORDER BY valor DESC,product_id) posicao
FROM t
)
SELECT category_id,product_id,valor,posicao FROM ranking WHERE posicao<=5 ORDER BY
category_id,posicao;
MongoDB:
```

```
// P07 – Top cinco produtos por categoria
// Pergunta: Quais são os cinco produtos de maior valor em cada categoria?
// Resultado: Até cinco produtos de maior valor em cada categoria, com posição 1 a 5 e desempate
pelo menor ID.
// Datas UTC; Decimal128; saída ordenada equivalente ao SQL.
var consulta = {
  collection: "orders",
  pipeline: [
    {
      $match: {
        order_date: {$gte: ISODate("1996-07-01T00:00:00Z"), $lt: ISODate("1998-06-01T00:00:00Z")}
      }
    },
    {$unwind: "$items"},
    {
      $group: {
        _id: {c: "$items.product.category_id", p: "$items.product._id"},
        valor: {
          $sum: {
            $multiply: [
              "$items.unit_price",
              "$items.quantity",
              {$subtract: [Decimal128("1"), "$items.discount"]}
            ]
          }
        }
      }
    },
    {$project: {_id: 0, category_id: "$_id.c", product_id: "$_id.p", valor: 1}},
    {
      $setWindowFields: {
        partitionBy: "$category_id",
        sortBy: {valor: -1, product_id: 1},
        output: {posicao: {$sum: 1, window: {documents: ["unbounded", "current"]}}}
      }
    },
    {$match: {posicao: {$lte: 5}}},
    {$sort: {category_id: 1, posicao: 1}}
  ]
};
if (typeof ANALYTICS_EXPORT === 'undefined') {
  print(EJSON.stringify(db.getSiblingDB('northwind')
    .getCollection(consulta.collection).aggregate(consulta.pipeline).toArray(), null, 2));
}
```

A.8 Par Q08/P08

PostgreSQL:

```
-- Q08 – Valor mensal acumulado
-- Pergunta: Qual é o valor acumulado ao longo dos meses?
-- Resultado: Valor de cada mês e o valor acumulado desde julho de 1996; o último acumulado é o
total do período.
-- Intervalo [1996-07-01, 1998-06-01]; RFM/inatividade: 1998-05-06.
WITH calendario AS (
  SELECT generate_series(DATE '1996-07-01', DATE '1998-06-01'-INTERVAL '1 month', INTERVAL '1
month')::date mes
), totais AS (
  SELECT date_trunc('month',order_date)::date mes, count(*) pedidos, sum(valor) valor
  FROM nw_analytics.vw_pedido_valor WHERE order_date >= DATE '1996-07-01' AND order_date < DATE
'1998-06-01' GROUP BY 1
), mensal AS (
  SELECT c.mes,coalesce(t.pedidos,0) pedidos,coalesce(t.valor,0) valor
  FROM calendario c LEFT JOIN totais t USING(mes)
)
```

```
SELECT mes,valor,sum(valor) OVER (ORDER BY mes ROWS UNBOUNDED PRECEDING) acumulado FROM mensal
ORDER BY mes;
```

MongoDB:

```
// P08 – Valor mensal acumulado
// Pergunta: Qual é o valor acumulado ao longo dos meses?
// Resultado: Valor de cada mês e o valor acumulado desde julho de 1996; o último acumulado é o
// total do período.
// Datas UTC; Decimal128; saída ordenada equivalente ao SQL.
var consulta = {
  collection: "orders",
  pipeline: [
    {
      $match: {
        order_date: {$gte: ISODate("1996-07-01T00:00:00Z"), $lt: ISODate("1998-06-01T00:00:00Z")}
      }
    },
    {
      $set: {
        valor: {
          $sum: {
            $map: {
              input: "$items",
              as: "i",
              in: {
                $multiply: [
                  "$$i.unit_price",
                  "$$i.quantity",
                  {$subtract: [Decimal128("1"), "$$i.discount"]}
                ]
              }
            }
          }
        }
      }
    },
    {
      $facet: {
        dados: [
          {
            $group: {
              _id: {$dateTrunc: {date: "$order_date", unit: "month", timezone: "UTC"}},
              pedidos: {$sum: 1},
              valor: {$sum: "$valor"}
            }
          ]
        ]
      }
    },
    {
      $project: {
        linhas: {
          $map: {
            input: {
              $range: [
                0,
                {
                  $dateDiff: {
                    startDate: ISODate("1996-07-01T00:00:00Z"),
                    endDate: ISODate("1998-06-01T00:00:00Z"),
                    unit: "month"
                  }
                }
              ]
            },
            as: "k",
            in: {
              $let: {
                vars: {
                  m: {
                    $dateAdd: {
                      startDate: ISODate("1996-07-01T00:00:00Z"),
                      unit: "month",
                      amount: "$$k"
                    }
                  }
                },
                in: {
                  $let: {
                    vars: {
                      r: {

```



```

    order_date: {$gte: ISODate("1996-07-01T00:00:00Z"), $lt: ISODate("1998-06-01T00:00:00Z")}
  },
  {
    $set: {
      valor: {
        $sum: {
          $map: {
            input: "$items",
            as: "i",
            in: {
              $multiply: [
                "$$i.unit_price",
                "$$i.quantity",
                {$subtract: [Decimal128("1"), "$$i.discount"]}
              ]
            }
          }
        }
      }
    }
  },
  {
    $facet: {
      dados: [
        {
          $group: {
            _id: {$dateTrunc: {date: "$order_date", unit: "month", timezone: "UTC"}},
            pedidos: {$sum: 1},
            valor: {$sum: "$valor"}
          }
        }
      ]
    }
  },
  {
    $project: {
      linhas: {
        $map: {
          input: {
            $range: [
              0,
              {
                $dateDiff: {
                  startDate: ISODate("1996-07-01T00:00:00Z"),
                  endDate: ISODate("1998-06-01T00:00:00Z"),
                  unit: "month"
                }
              }
            ]
          },
          as: "k",
          in: {
            $let: {
              vars: {
                m: {
                  $dateAdd: {
                    startDate: ISODate("1996-07-01T00:00:00Z"),
                    unit: "month",
                    amount: "$$k"
                  }
                }
              }
            },
            in: {
              $let: {
                vars: {
                  r: {
                    $arrayElemAt: [
                      {
                        $filter: {input: "$dados", as: "d", cond: {$eq: ["$$d._id", "$$m"]}}
                      },
                      0
                    ]
                  }
                },
                in: {
                  mes: "$$m",
                  pedidos: {$ifNull: ["$$r.pedidos", 0]},
                  valor: {$ifNull: ["$$r.valor", Decimal128("0")]}
                }
              }
            }
          }
        }
      }
    }
  }
}

```

```

    }
  }
}
},
{$unwind: "$linhas"},
{$replaceWith: "$linhas"},
{
  $setWindowFields: {
    sortBy: {mes: 1},
    output: {anterior: {$shift: {output: "$valor", by: -1, default: null}}}
  }
},
{$set: {variacao: {$subtract: ["$valor", "$anterior"]}}},
{
  $project: {
    _id: 0,
    mes: {$dateToString: {date: "$mes", format: "%Y-%m-%d"}},
    valor: 1,
    anterior: 1,
    variacao: 1,
    percentual: {
      $cond: [
        {$eq: ["$anterior", 0]},
        null,
        {$divide: [{$multiply: ["$variacao", Decimal128("100")]}, "$anterior"]}
      ]
    }
  }
},
{$sort: {mes: 1}}
]
};
if (typeof ANALYTICS_EXPORT === 'undefined') {
  print(EJSON.stringify(db.getSiblingDB('northwind')
    .getCollection(consulta.collection).aggregate(consulta.pipeline).toArray(), null, 2));
}

```

A.10 Par Q10/P10

PostgreSQL:

```

-- Q10 – Curva ABC de clientes
-- Pergunta: Como os clientes se distribuem pela curva ABC de valor?
-- Resultado: Todos os clientes ordenados por valor, com acumulado e classe A/B/C pela
concentração do valor.
-- Intervalo [1996-07-01, 1998-06-01); RFM/inatividade: 1998-05-06.
WITH t AS (
  SELECT c.customer_id, c.company_name cliente, count(o.order_id) pedidos, coalesce(sum(o.valor), 0)
  valor
  FROM nw.customers c LEFT JOIN nw_analytics.vw_pedido_valor o
  ON o.customer_id=c.customer_id AND order_date >= DATE '1996-07-01' AND order_date < DATE '1998-
06-01' GROUP BY c.customer_id
), a AS (
  SELECT *, sum(valor) OVER () total,
  sum(valor) OVER (ORDER BY valor DESC, customer_id ROWS UNBOUNDED PRECEDING) acumulado FROM t
)
SELECT customer_id, cliente, pedidos, valor, acumulado,
CASE WHEN pedidos=0 THEN 'sem compras'
WHEN acumulado-valor<total*0.80 THEN 'A'
WHEN acumulado-valor<total*0.95 THEN 'B' ELSE 'C' END classe
FROM a ORDER BY valor DESC, customer_id;

```

MongoDB:

```

// P10 – Curva ABC de clientes
// Pergunta: Como os clientes se distribuem pela curva ABC de valor?
// Resultado: Todos os clientes ordenados por valor, com acumulado e classe A/B/C pela
concentração do valor.
// Datas UTC; Decimal128; saída ordenada equivalente ao SQL.
var consulta = {
  collection: "customers",
  pipeline: [
    {
      $lookup: {
        from: "orders",
        localField: "_id",
        foreignField: "customer_id",
        pipeline: [

```

```

    $match: {
      order_date: {
        $gte: ISODate("1996-07-01T00:00:00Z"),
        $lt: ISODate("1998-06-01T00:00:00Z")
      }
    },
  },
  {
    $group: {
      _id: null,
      pedidos: {$sum: 1},
      valor: {
        $sum: {
          $sum: {
            $map: {
              input: "$items",
              as: "i",
              in: {
                $multiply: [
                  "$$i.unit_price",
                  "$$i.quantity",
                  {$subtract: [Decimal128("1"), "$$i.discount"]}
                ]
              }
            }
          }
        }
      }
    }
  }
},
],
as: "compras"
}
},
{
  $set: {
    pedidos: {$ifNull: [{ $first: "$compras.pedidos"}, 0]},
    valor: {$ifNull: [{ $first: "$compras.valor"}, Decimal128("0")]}
  }
},
{
  $setWindowFields: {
    sortBy: {valor: -1, _id: 1},
    output: {
      acumulado: {$sum: "$valor", window: {documents: ["unbounded", "current"]}},
      total: {$sum: "$valor", window: {documents: ["unbounded", "unbounded"]}}
    }
  }
},
{
  $project: {
    _id: 0,
    customer_id: "$_id",
    cliente: "$company_name",
    pedidos: 1,
    valor: 1,
    acumulado: 1,
    classe: {
      $switch: {
        branches: [
          {case: {$eq: ["$pedidos", 0]}, then: "sem compras"},
          {
            case: {
              $lt: [
                {$subtract: ["$acumulado", "$valor"]},
                {$multiply: ["$total", Decimal128("0.80")]}
              ]
            },
            then: "A"
          },
          {
            case: {
              $lt: [
                {$subtract: ["$acumulado", "$valor"]},
                {$multiply: ["$total", Decimal128("0.95")]}
              ]
            },
            then: "B"
          }
        ],
        default: "C"
      }
    }
  }
}

```

```

    }
  }
},
{$sort: {valor: -1, customer_id: 1}}
]
};
if (typeof ANALYTICS_EXPORT === 'undefined') {
  print(EJSON.stringify(db.getSiblingDB('northwind')
    .getCollection(consulta.collection).aggregate(consulta.pipeline).toArray(), null, 2));
}

```

A.11 Par Q11/P11

PostgreSQL:

```

-- Q11 – RFM descritivo dos clientes
-- Pergunta: Qual é o perfil de recência, frequência e valor de cada cliente?
-- Resultado: Recência (dias), frequência (pedidos) e valor de cada cliente, com escores 0 a 3 em faixas fixas.
-- Intervalo [1996-07-01, 1998-06-01]; RFM/inatividade: 1998-05-06.
WITH rfm AS (
  SELECT c.customer_id, DATE '1998-05-06' - max(o.order_date) recencia,
         count(o.order_id) frequencia, coalesce(sum(o.valor), 0) valor
  FROM nw.customers c LEFT JOIN nw_analytics.vw_pedido_valor o
  ON o.customer_id = c.customer_id AND o.order_date <= DATE '1998-05-06'
  GROUP BY c.customer_id
)
SELECT *, CASE WHEN frequencia = 0 THEN 0 WHEN recencia <= 30 THEN 3 WHEN recencia <= 90 THEN 2 ELSE 1
END r,
         CASE WHEN frequencia = 0 THEN 0 WHEN frequencia >= 10 THEN 3 WHEN frequencia >= 5 THEN 2 ELSE 1 END f,
         CASE WHEN frequencia = 0 THEN 0 WHEN valor >= 10000 THEN 3 WHEN valor >= 5000 THEN 2 ELSE 1 END m
FROM rfm ORDER BY customer_id;

```

MongoDB:

```

// P11 – RFM descritivo dos clientes
// Pergunta: Qual é o perfil de recência, frequência e valor de cada cliente?
// Resultado: Recência (dias), frequência (pedidos) e valor de cada cliente, com escores 0 a 3 em faixas fixas.
// Datas UTC; Decimal128; saída ordenada equivalente ao SQL.
var consulta = {
  collection: "customers",
  pipeline: [
    {
      $lookup: {
        from: "orders",
        localField: "_id",
        foreignField: "customer_id",
        pipeline: [
          { $match: { order_date: { $lte: ISODate("1998-05-06T00:00:00Z") } } },
          {
            $group: {
              _id: null,
              ultima: { $max: "$order_date" },
              frequencia: { $sum: 1 },
              valor: {
                $sum: {
                  $sum: {
                    $map: {
                      input: "$items",
                      as: "i",
                      in: {
                        $multiply: [
                          "$$i.unit_price",
                          "$$i.quantity",
                          { $subtract: [Decimal128("1"), "$$i.discount"] }
                        ]
                      }
                    }
                  }
                }
              }
            }
          }
        ]
      },
      as: "compras"
    },
    {
      $set: {
        ultima: { $ifNull: [{ $first: "$compras.ultima" }, null ] },

```

```

        frecuencia: {$ifNull: [{ $first: "$compras.frecuencia"}, 0]},
        valor: {$ifNull: [{ $first: "$compras.valor"}, Decimal128("0")]}
    },
    {
        $set: {
            recencia: {
                $dateDiff: {startDate: "$ultima", endDate: ISODate("1998-05-06T00:00:00Z"), unit:
"day"}
            }
        }
    },
    {
        $project: {
            _id: 0,
            customer_id: "$_id",
            recencia: 1,
            frecuencia: 1,
            valor: 1,
            r: {
                $cond: [
                    {$eq: ["$frecuencia", 0]},
                    0,
                    {
                        $cond: [{ $lte: ["$recencia", 30]}, 3, {$cond: [{ $lte: ["$recencia", 90]}, 2, 1]}
                    ]
                ]
            },
            f: {
                $cond: [
                    {$eq: ["$frecuencia", 0]},
                    0,
                    {
                        $cond: [
                            {$gte: ["$frecuencia", 10]},
                            3,
                            {$cond: [{ $gte: ["$frecuencia", 5]}, 2, 1]}
                        ]
                    }
                ]
            },
            m: {
                $cond: [
                    {$eq: ["$frecuencia", 0]},
                    0,
                    {
                        $cond: [
                            {$gte: ["$valor", Decimal128("10000")]},
                            3,
                            {$cond: [{ $gte: ["$valor", Decimal128("5000")]}, 2, 1]}
                        ]
                    }
                ]
            }
        ]
    },
    {$sort: {customer_id: 1}}
];
if (typeof ANALYTICS_EXPORT === 'undefined') {
    print(EJSON.stringify(db.getSiblingDB('northwind')
        .getCollection(consulta.collection).aggregate(consulta.pipeline).toArray(), null, 2));
}

```

A.12 Par Q12/P12

PostgreSQL:

```

-- Q12 – Produtos comprados juntos
-- Pergunta: Quais produtos aparecem juntos no mesmo pedido?
-- Resultado: Pares de produtos que aparecem no mesmo pedido, quantos pedidos contêm cada par e o
suporte (%).
-- Intervalo [1996-07-01, 1998-06-01); RFM/inatividade: 1998-05-06.
WITH pares AS (
    SELECT a.product_id produto_a, b.product_id produto_b, count(*) pedidos
    FROM nw.orders o JOIN nw.order_items a USING(order_id)
    JOIN nw.order_items b ON b.order_id=a.order_id AND a.product_id<b.product_id
    WHERE order_date >= DATE '1996-07-01' AND order_date < DATE '1998-06-01' GROUP BY
a.product_id, b.product_id
), total AS (SELECT count(*) denominador FROM nw.orders WHERE order_date >= DATE '1996-07-01' AND
order_date < DATE '1998-06-01')

```

```
SELECT pares.*,total.denominador,100.0*pedidos/nullif(denominador,0) suporte
FROM pares CROSS JOIN total ORDER BY pedidos DESC,produto_a,produto_b;
```

MongoDB:

```
// P12 – Produtos comprados juntos
// Pergunta: Quais produtos aparecem juntos no mesmo pedido?
// Resultado: Pares de produtos que aparecem no mesmo pedido, quantos pedidos contêm cada par e o
// suporte (%).
// Datas UTC; Decimal128; saída ordenada equivalente ao SQL.
var consulta = {
  collection: "orders",
  pipeline: [
    {
      $match: {
        order_date: {$gte: ISODate("1996-07-01T00:00:00Z"), $lt: ISODate("1998-06-01T00:00:00Z")}
      }
    },
    {
      $facet: {
        total: [{$count: "n"}],
        pares: [
          {$project: {_id: 0, ids: "$items.product._id"}},
          {$set: {outros: "$ids"}},
          {$unwind: "$ids"},
          {$unwind: "$outros"},
          {$match: {$expr: {$lt: ["$ids", "$outros"]}}},
          {$group: {_id: {a: "$ids", b: "$outros"}, pedidos: {$sum: 1}}}
        ]
      }
    },
    {$set: {denominador: {$ifNull: [{$first: "$total.n"}, 0]}}},
    {$unwind: "$pares"},
    {
      $project: {
        _id: 0,
        produto_a: "$pares._id.a",
        produto_b: "$pares._id.b",
        pedidos: "$pares.pedidos",
        denominador: 1,
        suporte: {
          $cond: [
            {$eq: ["$denominador", 0]},
            null,
            {
              $divide: [{$multiply: ["$pares.pedidos", Decimal128("100")]}, "$denominador"]
            }
          ]
        }
      }
    },
    {$sort: {pedidos: -1, produto_a: 1, produto_b: 1}}
  ]
};
if (typeof ANALYTICS_EXPORT === 'undefined') {
  print(EJSON.stringify(db.getSiblingDB('northwind')
    .getCollection(consulta.collection).aggregate(consulta.pipeline).toArray(), null, 2));
}
```

A.13 Par Q13/P13

PostgreSQL:

```
-- Q13 – Intervalo até envio por transportadora
-- Pergunta: Qual é o intervalo até o envio registrado por transportadora?
-- Resultado: Por transportadora: pedidos, datas de envio registradas e ausentes, média de dias
-- até o envio e envios após a data requerida.
-- Intervalo [1996-07-01, 1998-06-01]; RFM/inatividade: 1998-05-06.
SELECT s.shipper_id,s.company_name transportadora,count(o.order_id) pedidos,
count(o.shipped_date) com_envio,count(o.order_id)-count(o.shipped_date) sem_data,
coalesce(sum(o.shipped_date-o.order_date),0) soma_dias,
avg(o.shipped_date-o.order_date) media_dias,
count(*) FILTER (WHERE o.shipped_date>o.required_date) apos_requerida
FROM nw.shippers s LEFT JOIN nw.orders o ON o.shipper_id=s.shipper_id AND order_date >= DATE
'1996-07-01' AND order_date < DATE '1998-06-01'
GROUP BY s.shipper_id ORDER BY s.shipper_id;
```

MongoDB:

```
// P13 – Intervalo até envio por transportadora
// Pergunta: Qual é o intervalo até o envio registrado por transportadora?
// Resultado: Por transportadora: pedidos, datas de envio registradas e ausentes, média de dias
-- até o envio e envios após a data requerida.
```

```
// Datas UTC; Decimal128; saída ordenada equivalente ao SQL.
var consulta = {
  collection: "shippers",
  pipeline: [
    {
      $lookup: {
        from: "orders",
        localField: "_id",
        foreignField: "shipper_id",
        pipeline: [
          {
            $match: {
              order_date: {
                $gte: ISODate("1996-07-01T00:00:00Z"),
                $lt: ISODate("1998-06-01T00:00:00Z")
              }
            }
          },
          {
            $group: {
              _id: null,
              pedidos: {$sum: 1},
              com_envio: {$sum: {$cond: [{ $ne: ["$shipped_date", null]}, 1, 0]}},
              soma_dias: {
                $sum: {
                  $dateDiff: {startDate: "$order_date", endDate: "$shipped_date", unit: "day"}
                }
              },
              apos_requerida: {$sum: {$cond: [{ $gt: ["$shipped_date", "$required_date"]}, 1, 0]}}
            }
          }
        ],
        as: "res"
      }
    },
    {
      $set: {
        pedidos: {$ifNull: [{ $first: "$res.pedidos"}, 0]},
        com_envio: {$ifNull: [{ $first: "$res.com_envio"}, 0]},
        soma_dias: {$ifNull: [{ $first: "$res.soma_dias"}, 0]},
        apos_requerida: {$ifNull: [{ $first: "$res.apos_requerida"}, 0]}
      }
    },
    {
      $project: {
        _id: 0,
        shipper_id: "$_id",
        transportadora: "$company_name",
        pedidos: 1,
        com_envio: 1,
        sem_data: {$subtract: ["$pedidos", "$com_envio"]},
        soma_dias: 1,
        apos_requerida: 1,
        media_dias: {
          $cond: [
            { $eq: ["$com_envio", 0]},
            null,
            { $divide: [{ $toDecimal: "$soma_dias"}, "$com_envio"]}
          ]
        }
      }
    },
    { $sort: {shipper_id: 1}}
  ]
};
if (typeof ANALYTICS_EXPORT === 'undefined') {
  print(EJSON.stringify(db.getSiblingDB('northwind')
    .getCollection(consulta.collection).aggregate(consulta.pipeline).toArray(), null, 2));
}
```

A.14 Par Q14/P14

PostgreSQL:

```
-- Q14 – Descontos sobre o valor dos pedidos
-- Pergunta: Quanto os descontos reduzem o valor bruto dos itens?
-- Resultado: Por categoria: valor bruto, desconto concedido, valor após descontos e taxa
ponderada de desconto.
-- Intervalo [1996-07-01, 1998-06-01]; RFM/inatividade: 1998-05-06.
SELECT category_id,sum(bruto) bruto,sum(desconto) desconto,sum(valor) valor,
100*sum(desconto)/nullif(sum(bruto),0) percentual_desconto
```

```
FROM nw_analytics.vw_item_valor WHERE order_date >= DATE '1996-07-01' AND order_date < DATE
'1998-06-01'
GROUP BY category_id ORDER BY category_id;
```

MongoDB:

```
// P14 – Descontos sobre o valor dos pedidos
// Pergunta: Quanto os descontos reduzem o valor bruto dos itens?
// Resultado: Por categoria: valor bruto, desconto concedido, valor após descontos e taxa
ponderada de desconto.
// Datas UTC; Decimal128; saída ordenada equivalente ao SQL.
var consulta = {
  collection: "orders",
  pipeline: [
    {
      $match: {
        order_date: {$gte: ISODate("1996-07-01T00:00:00Z"), $lt: ISODate("1998-06-01T00:00:00Z")}
      }
    },
    {$unwind: "$items"},
    {
      $set: {
        bruto: {$multiply: ["$items.unit_price", "$items.quantity"]},
        desconto: {$multiply: ["$items.unit_price", "$items.quantity", "$items.discount"]},
        valor: {
          $multiply: [
            "$items.unit_price",
            "$items.quantity",
            {$subtract: [Decimal128("1"), "$items.discount"]}
          ]
        }
      }
    },
    {
      $group: {
        _id: "$items.product.category._id",
        bruto: {$sum: "$bruto"},
        desconto: {$sum: "$desconto"},
        valor: {$sum: "$valor"}
      }
    },
    {
      $project: {
        _id: 0,
        category_id: "$_id",
        bruto: 1,
        desconto: 1,
        valor: 1,
        percentual_desconto: {
          $cond: [
            {$eq: ["$bruto", 0]},
            null,
            {$divide: [{$multiply: ["$desconto", Decimal128("100")]}, "$bruto"]}
          ]
        }
      }
    },
    {$sort: {category_id: 1}}
  ]
};
if (typeof ANALYTICS_EXPORT === 'undefined') {
  print(EJSON.stringify(db.getSiblingDB('northwind')
    .getCollection(consulta.collection).aggregate(consulta.pipeline).toArray(), null, 2));
}
```

A.15 Par Q15/P15

PostgreSQL:

```
-- Q15 – Vendas próprias e da equipe
-- Pergunta: Quanto foi registrado por cada funcionário e sua equipe?
-- Resultado: Por funcionário: tamanho da equipe (ele e subordinados), valor próprio e valor da
equipe; equipes se sobrepõem.
-- Intervalo [1996-07-01, 1998-06-01]; RFM/inatividade: 1998-05-06.
WITH RECURSIVE equipe AS (
  SELECT employee_id gestor_id, employee_id membro_id, ARRAY[employee_id] caminho FROM nw.employees
  UNION ALL
  SELECT q.gestor_id, e.employee_id, q.caminho || e.employee_id
  FROM equipe q JOIN nw.employees e ON e.reports_to=q.membro_id
  WHERE NOT e.employee_id=ANY(q.caminho)
), vendas AS (
  SELECT employee_id, count(*) pedidos, sum(valor) valor
```

```

FROM nw_analytics.vw_pedido_valor WHERE order_date >= DATE '1996-07-01' AND order_date < DATE
'1998-06-01' GROUP BY employee_id
)
SELECT q.gestor_id, count(*) membros,
  coalesce(sum(v.valor) FILTER (WHERE q.membro_id=q.gestor_id),0) valor proprio,
  coalesce(sum(v.pedidos),0) pedidos equipe, coalesce(sum(v.valor),0) valor equipe
FROM equipe q LEFT JOIN vendas v ON v.employee_id=q.membro_id
GROUP BY q.gestor_id ORDER BY q.gestor_id;

```

MongoDB:

```

// P15 – Vendas próprias e da equipe
// Pergunta: Quanto foi registrado por cada funcionário e sua equipe?
// Resultado: Por funcionário: tamanho da equipe (ele e subordinados), valor próprio e valor da
equipe; equipes se sobrepõem.
// Datas UTC; Decimal128; saída ordenada equivalente ao SQL.
var consulta = {
  collection: "employees",
  pipeline: [
    {
      $graphLookup: {
        from: "employees",
        startWith: "$_id",
        connectFromField: "_id",
        connectToField: "reports_to",
        as: "subordinados"
      }
    },
    {
      $set: {membro_ids: {$setUnion: [["_id"], "$subordinados._id"]}},
    },
    {
      $lookup: {
        from: "orders",
        localField: "membro_ids",
        foreignField: "employee_id",
        pipeline: [
          {
            $match: {
              order_date: {
                $gte: ISODate("1996-07-01T00:00:00Z"),
                $lt: ISODate("1998-06-01T00:00:00Z")
              }
            }
          },
          {
            $group: {
              _id: "$employee_id",
              pedidos: {$sum: 1},
              valor: {
                $sum: {
                  $sum: {
                    $map: {
                      input: "$items",
                      as: "i",
                      in: {
                        $multiply: [
                          "$$i.unit_price",
                          "$$i.quantity",
                          {$subtract: [Decimal128("1"), "$$i.discount"]}
                        ]
                      }
                    }
                  }
                }
              }
            }
          }
        ]
      }
    },
    {
      as: "vendas"
    }
  ],
  {
    $project: {
      _id: 0,
      gestor_id: "$_id",
      membros: {$size: "$membro_ids"},
      valor proprio: {
        $sum: {
          $map: {
            input: {$filter: {input: "$vendas", as: "v", cond: {$eq: [ "$$v._id", "$_id" ]}}},
            as: "v",
            in: "$$v.valor"
          }
        }
      }
    }
  }
}

```

```

    },
    pedidos_equipe: {$sum: "$vendas.pedidos"},
    valor_equipe: {$sum: "$vendas.valor"}
  }
},
{$sort: {gestor_id: 1}}
]
};
if (typeof ANALYTICS_EXPORT === 'undefined') {
  print(EJSON.stringify(db.getSiblingDB('northwind')
    .getCollection(consulta.collection).aggregate(consulta.pipeline).toArray(), null, 2));
}

```

A.16 Par Q16/P16

PostgreSQL:

```

-- Q16 – Distribuição trimestral por ano e categoria
-- Pergunta: Como o valor se distribui por ano, trimestre e categoria?
-- Resultado: Valor por ano e trimestre, total e por categoria, marcando trimestres com cobertura
parcial dos dados.
-- Intervalo [1996-07-01, 1998-06-01); RFM/inatividade: 1998-05-06.
WITH t AS (
  SELECT extract(year FROM order_date)::int ano,
         extract(quarter FROM order_date)::int trimestre, category_id, valor
  FROM nw_analytics.vw_item_valor WHERE order_date >= DATE '1996-07-01' AND order_date < DATE
'1998-06-01'
)
SELECT ano, trimestre, category_id, sum(valor) valor,
       CASE WHEN grouping(category_id)=1 THEN 'total' ELSE 'categoria' END nivel,
       (make_date(ano, (trimestre-1)*3+1, 1) < DATE '1996-07-04' OR
        (make_date(ano, (trimestre-1)*3+1, 1) + INTERVAL '3 months')::date > DATE '1998-05-07') parcial
FROM t GROUP BY GROUPING SETS ((ano, trimestre, category_id), (ano, trimestre))
ORDER BY ano, trimestre, category_id NULLS FIRST;

```

MongoDB:

```

// P16 – Distribuição trimestral por ano e categoria
// Pergunta: Como o valor se distribui por ano, trimestre e categoria?
// Resultado: Valor por ano e trimestre, total e por categoria, marcando trimestres com cobertura
parcial dos dados.
// Datas UTC; Decimal128; saída ordenada equivalente ao SQL.
var consulta = {
  collection: "orders",
  pipeline: [
    {
      $match: {
        order_date: {$gte: ISODate("1996-07-01T00:00:00Z"), $lt: ISODate("1998-06-01T00:00:00Z")}
      }
    },
    {$unwind: "$items"},
    {
      $set: {
        ano: {$year: "$order_date"},
        trimestre: {$toInt: {$ceil: {$divide: [{$month: "$order_date"}, 3]}}},
        valor: {
          $multiply: [
            "$items.unit_price",
            "$items.quantity",
            {$subtract: [Decimal128("1"), "$items.discount"]}
          ]
        }
      }
    }
  ],
  $facet: {
    total: [
      {$group: {_id: {ano: "$ano", trimestre: "$trimestre"}, valor: {$sum: "$valor"}}},
      {
        $project: {
          _id: 0,
          ano: "$_id.ano",
          trimestre: "$_id.trimestre",
          category_id: {$literal: null},
          valor: 1,
          nivel: {$literal: "total"}
        }
      }
    ],
    categoria: [
      {

```

```

        $group: {
          _id: {
            ano: "$ano",
            trimestre: "$trimestre",
            category_id: "$items.product.category._id"
          },
          valor: {$sum: "$valor"}
        },
      },
    {
      $project: {
        _id: 0,
        ano: "$_id.ano",
        trimestre: "$_id.trimestre",
        category_id: "$_id.category_id",
        valor: 1,
        nivel: {$literal: "categoria"}
      }
    }
  ]
},
{$project: {linhas: {$concatArrays: ["$total", "$categoria"]}}},
{$unwind: "$linhas"},
{$replaceWith: "$linhas"},
{
  $set: {
    inicio_trimestre: {
      $dateFromParts: {
        year: "$ano",
        month: {$add: [{$multiply: [{$subtract: ["$trimestre", 1]}, 3]}, 1]}
      }
    }
  }
},
{
  $set: {
    parcial: {
      $or: [
        {$lt: ["$inicio_trimestre", ISODate("1996-07-04T00:00:00Z")]},
        {
          $gt: [
            {$dateAdd: {startDate: "$inicio_trimestre", unit: "month", amount: 3}},
            ISODate("1998-05-07T00:00:00Z")
          ]
        }
      ]
    }
  }
},
{$unset: "inicio_trimestre"},
{$sort: {ano: 1, trimestre: 1, category_id: 1}}
]
};
if (typeof ANALYTICS_EXPORT === 'undefined') {
  print(EJSON.stringify(db.getSiblingDB('northwind')
    .getCollection(consulta.collection).aggregate(consulta.pipeline).toArray(), null, 2));
}

```

Apêndice B — Views, procedures e recursos documentais

00_views.sql

```
-- Camada da Entrega 3. Não altera as views históricas nem os dados de nw.
CREATE SCHEMA IF NOT EXISTS nw_analytics;

CREATE OR REPLACE VIEW nw_analytics.vw_item_valor AS
SELECT i.order_id, i.product_id, o.order_date, o.customer_id,
       o.employee_id, p.category_id, i.quantity,
       i.unit_price * i.quantity AS bruto,
       i.unit_price * i.quantity * i.discount AS desconto,
       i.unit_price * i.quantity * (1-i.discount) AS valor
FROM nw.order_items i
JOIN nw.orders o USING (order_id)
JOIN nw.products p USING (product_id);

CREATE OR REPLACE VIEW nw_analytics.vw_pedido_valor AS
SELECT o.order_id, o.order_date, o.customer_id, o.employee_id,
       o.shipper_id, o.required_date, o.shipped_date,
       count(i.product_id) AS itens,
       coalesce(sum(i.quantity),0) AS unidades,
       coalesce(sum(i.unit_price*i.quantity*(1-i.discount)),0) AS valor
FROM nw.orders o LEFT JOIN nw.order_items i USING (order_id)
GROUP BY o.order_id;

CREATE OR REPLACE VIEW nw_analytics.vw_vendas_mensais AS
SELECT date_trunc('month',order_date)::date AS mes,
       count(*) AS pedidos, sum(valor) AS valor,
       sum(valor)/count(*) AS ticket
FROM nw_analytics.vw_pedido_valor GROUP BY 1;

CREATE OR REPLACE VIEW nw_analytics.vw_envio_transportadora AS
SELECT s.shipper_id, s.company_name AS transportadora,
       count(o.order_id) AS pedidos,
       count(o.shipped_date) AS com_envio,
       count(o.order_id)-count(o.shipped_date) AS sem_data,
       coalesce(sum(o.shipped_date-o.order_date),0) AS soma_dias,
       avg(o.shipped_date-o.order_date) AS media_dias,
       count(*) FILTER (WHERE o.shipped_date>o.required_date) AS apos_requerida
FROM nw.shippers s LEFT JOIN nw.orders o USING (shipper_id)
GROUP BY s.shipper_id;

COMMENT ON VIEW nw_analytics.vw_item_valor IS
'Valor exato após descontos, sem frete e sem arredondamento por item.';
COMMENT ON VIEW nw_analytics.vw_pedido_valor IS
'Uma linha por pedido; pedidos sem itens aparecem com valor zero.';
COMMENT ON VIEW nw_analytics.vw_vendas_mensais IS
'Meses com movimento; a consulta temporal completa o calendário do recorte.';
COMMENT ON VIEW nw_analytics.vw_envio_transportadora IS
'Envio registrado não comprova entrega; data nula significa ausência de registro.';
```

10_procedures.sql

```
-- Relatórios tabulares: abrir transação, CALL, FETCH e COMMIT.
CREATE OR REPLACE PROCEDURE nw_analytics.sp_resumo_vendas_periodo(
    IN inicio date, IN fim date, INOUT resultado refcursor)
LANGUAGE plpgsql AS $$
BEGIN
    IF inicio IS NULL OR fim IS NULL OR inicio>=fim THEN
        RAISE EXCEPTION 'Informe início anterior ao fim exclusivo';
    END IF;
    OPEN resultado FOR
    SELECT i.category_id,c.category_name categoria,
           count(DISTINCT i.order_id) pedidos,sum(i.valor) valor,
           sum(i.valor)/count(DISTINCT i.order_id) ticket_categoria
    FROM nw_analytics.vw_item_valor i
    JOIN nw.categories c USING(category_id)
    WHERE i.order_date>=inicio AND i.order_date<fim
    GROUP BY i.category_id,c.category_name ORDER BY i.category_id;
END;
$$;

CREATE OR REPLACE PROCEDURE nw_analytics.sp_clientes_inativos(
    IN referencia date, IN meses integer, INOUT resultado refcursor)
LANGUAGE plpgsql AS $$
BEGIN
    IF referencia IS NULL OR meses IS NULL OR meses<1 THEN
        RAISE EXCEPTION 'Informe referência e meses positivos';
    END IF;
    OPEN resultado FOR
    SELECT o.customer_id,c.customer_name cliente,
           count(o.order_id) pedidos,sum(o.valor) valor,
           sum(o.valor)/count(o.order_id) ticket_cliente
    FROM nw.orders o
    JOIN nw.customers c USING(customer_id)
    WHERE o.order_date>=referencia AND o.order_date<referencia+meses
    GROUP BY o.customer_id,c.customer_name ORDER BY o.customer_id;
```

```

END IF;
OPEN resultado FOR
WITH historico AS (
    SELECT c.customer_id,c.company_name cliente,max(o.order_date) ultima
    FROM nw.customers c LEFT JOIN nw.orders o
    ON o.customer_id=c.customer_id AND o.order_date<=referencia
    GROUP BY c.customer_id
)
SELECT *,referencia-ultima recencia,
    CASE WHEN ultima IS NULL THEN 'sem compras' ELSE 'inativo' END situacao
FROM historico
WHERE ultima IS NULL OR ultima<(referencia-make_interval(months=>meses))::date
ORDER BY customer_id;
END;
$$;

CREATE OR REPLACE PROCEDURE nw_analytics.sp_desempenho_equipe(
    IN gestor integer, IN inicio date, IN fim date, INOUT resultado refcursor)
LANGUAGE plpgsql AS $$
BEGIN
    IF inicio IS NULL OR fim IS NULL OR inicio>=fim THEN
        RAISE EXCEPTION 'Informe início anterior ao fim exclusivo';
    END IF;
    IF gestor IS NULL OR NOT EXISTS (
        SELECT 1 FROM nw.employees WHERE employee_id=gestor
    ) THEN RAISE EXCEPTION 'Gestor inexistente'; END IF;
    OPEN resultado FOR
    WITH RECURSIVE equipe AS (
        SELECT employee_id,ARRAY[employee_id] caminho
        FROM nw.employees WHERE employee_id=gestor
        UNION ALL
        SELECT e.employee_id,q.caminho||e.employee_id
        FROM equipe q JOIN nw.employees e ON e.reports_to=q.employee_id
        WHERE NOT e.employee_id=ANY(q.caminho)
    ), vendas AS (
        SELECT employee_id,count(*) pedidos,sum(valor) valor
        FROM nw_analytics.vw_pedido_valor
        WHERE order_date>=inicio AND order_date<fim GROUP BY employee_id
    )
    SELECT gestor gestor_id,count(*) membros,
        coalesce(sum(v.valor) FILTER (WHERE q.employee_id=gestor),0) valor_proprio,
        coalesce(sum(v.pedidos),0) pedidos_equipe,
        coalesce(sum(v.valor),0) valor_equipe
    FROM equipe q LEFT JOIN vendas v USING(employee_id);
END;
$$;

```

11_exemplos_procedures.sql

```

BEGIN;
CALL nw_analytics.sp_resumo_vendas_periodo(
    '1996-07-01','1998-06-01','vendas');
FETCH ALL FROM vendas;
CALL nw_analytics.sp_clientes_inativos('1998-05-06',6,'inativos');
FETCH ALL FROM inativos;
CALL nw_analytics.sp_desempenho_equipe(2,'1996-07-01','1998-06-01','equipe');
FETCH ALL FROM equipe;
COMMIT;

```

recursos_documentais.js

```

var b = db.getSiblingDB('northwind');
// As três condições precisam ocorrer no MESMO elemento do array.
var mesmoItem = [
    { $match: { items: { $elemMatch: {
        'product._id': 11, quantity: { $gte: 10 }, discount: { $gt: Decimal128('0') }
    } } } },
    { $project: { _id: 1, items: { $filter: { input: '$items', as: 'i', cond: { $and: [
        { $eq: [ '$$i.product._id', 11 ] }, { $gte: [ '$$i.quantity', 10 ] },
        { $gt: [ '$$i.discount', Decimal128('0') ] }
    ] } } } } },
    { $sort: { _id: 1 } }
];
// Contraste: as mesmas condições escritas separadamente podem ser atendidas por itens
diferentes.
var independentes = { 'items.product._id': 11, 'items.quantity': { $gte: 10 },
    'items.discount': { $gt: Decimal128('0') } };
// Um documento de entrada continua sendo um documento de saída.
var pedido = [
    { $match: { _id: 10248 } },
    { $project: { _id: 1, produtos: { $map: { input: '$items', as: 'i', in: '$$i.product.name' } } },

```

```

    valor:{$reduce:{input:'$items',initialValue:Decimal128('0'),in:{$add:[
      '$$value',{multiply:[ '$$this.unit_price','$$this.quantity',
        {$subtract:[Decimal128('1'),'$$this.discount']}]}]}]}]}
  ];
// Facetas diferentes sobre os mesmos documentos de entrada.
var painel = [
  {$match:{order_date:{$gte:ISODate('1997-01-01'),$lt:ISODate('1998-01-01')}}},
  {$facet:{
    volume:[{$count:'pedidos'}],
    envio:[{$group:{_id:{$cond:[{$eq:['$shipped_date',null]],'sem data','com data'}},
      pedidos:{$sum:1}},{$sort:{_id:1}}],
    tamanho:[{$group:{_id:{$size:'$items'},pedidos:{$sum:1}},{$sort:{_id:1}}]
  }}
];
print(EJSON.stringify({mesmo_item:b.orders.aggregate(mesmoItem).toArray(),
  condicoes_independentes:b.orders.countDocuments(independentes),

pedido:b.orders.aggregate(pedido).toArray(),painel:b.orders.aggregate(painel).toArray()},null,2))
;

```

mapreduce.js

```

// MapReduce é requisito didático legado. Somente quantidades inteiras.
var b = db.getSiblingDB('northwind');
var resultado = b.runCommand({
  mapReduce: 'orders',
  map: function () {
    this.items.forEach(function (i) { emit(i.product._id, i.quantity); });
  },
  reduce: function (produto, quantidades) { return Array.sum(quantidades); },
  out: { inline: 1 }
});
if (resultado.ok !== 1) throw new Error(EJSON.stringify(resultado));
var pipeline = [
  {$unwind: '$items'},
  {$group: {_id: '$items.product._id', value: {$sum: '$items.quantity'}}},
  {$sort: {_id: 1}}
];
var moderno = b.orders.aggregate(pipeline).toArray();
var legado = resultado.results.sort((a,b) => a._id-b._id);
require('assert').deepStrictEqual(
  legado.map(x=>[Number(x._id),Number(x.value)]),
  moderno.map(x=>[Number(x._id),Number(x.value)])
);
print(EJSON.stringify({equivalente:true,produtos:moderno.length,
  unidades:moderno.reduce((s,x)=>s+Number(x.value),0),resultados:legado},null,2));

```